



6CCS3PRJ

**Optimising Architectural Space: A
Reinforcement Learning Approach for
Cost-Effective, Compliance-Aware Floor
Plan Design from a Real Estate
Perspective**

Final Project Report

Author: Heman Seegolam

Supervisor: Milad Kazemi Mehrabadi

Student ID: 21003628

Programme of Study: BSc Computer Science

April 4, 2025

Abstract

Architectural design necessitates an intricate balance of functionality, economic efficiency, and regulatory constraints. Real estate remains the global, leading form of wealth generation.

The value of this project is two-fold. It first aims to introduce a comprehensive real estate management application designed to restrict property oversight and enhance management processes through simplistic design. It additionally aims to leverage reinforcement learning techniques to address architectural challenges by creating an agent that, through iterative learning, optimises floor plan design, simultaneously maximising spatial utility, minimising costs and ensuring full compliance with legal, regulatory constraints.

By integrating machine learning with architectural design principles within a real estate context, this project demonstrates the potential for artificial intelligence-driven solutions to transform spatial planning across residential, commercial, and institutional contexts.

Originality Avowal

I verify that I am the sole author of this report, except where explicitly stated to the contrary. I grant the right to King's College London to make paper and electronic copies of the submitted work for purposes of marking, plagiarism detection and archival, and to upload a copy of the work to Turnitin or another trusted plagiarism detection service. I confirm this report does not exceed 25,000 words.

Heman Seegolam

April 4, 2025

Acknowledgements

I would like to give thanks to my project supervisor, Milad Kazemi Mehrabadi, for his guidance, support and supervision throughout the project.

Contents

1	Introduction	5
1.1	Motivations	5
1.2	Objectives	7
2	Background	8
2.1	Real Estate Legal Compliance	8
2.2	Reinforcement Learning (RL)	10
2.3	Security	15
3	Literature Review	19
3.1	ArchiGAN	19
3.2	SpaceLayoutGym	21
4	Requirements & Specification	23
4.1	Requirements	23
4.2	Specification	26
5	Design	31
5.1	System Architecture	31
5.2	Use Cases	32
5.3	System Structure & Data Management	33
5.4	User Interface	34
5.5	Security	43
5.6	Using Reinforcement Learning	45
6	Implementation & Testing	47
6.1	Methodology	47
6.2	Code Organisation	48
6.3	System Functionality	54
7	Legal, Social, Ethical and Professional Issues	87
7.1	Legal and Ethical Issues	87
7.2	Social and Professional Issues	88
7.3	Real Estate Legal Compliance	88

8	Evaluation	90
8.1	PPO Agent	90
9	Conclusion	94
9.1	Closing Remarks	94
9.2	Future Work	95
	Bibliography	102
A	Extra Information	103
B	User Guide	107
B.1	Installation	107

Acronyms

ANN Artificial Neural Network. 10, 11

API Application Programming Interface. 17, 28, 30, 45, 51, 52, 55

BCS British Computer Society. 87, 88

CNN Convolutional Neural Networks. 46, 93

CORS Cross-Origin Resource Sharing. 18, 44, 55

CSRF Cross-Site Request Forgery. 18, 44, 56

CSS Cascading Style Sheets. 48, 49, 52

DOM Document Object Model. 26, 50

DOS Denial-of-Service. 15, 17, 43, 55

ES ECMAScript. 27

GAE Generalised Advantage Estimation. 14, 77, 78

GAN Generative Adversarial Network. 19, 22

GDPR General Data Protection Regulation. 88

GEOS Geometry Engine, Open Source. 29

GIS Geographic Information System. 29

HMO Houses in Multiple Occupations. 8, 9

HSTS HTTP Strict Transport Security. 43

HTML HyperText Markup Language. 50, 52

HTTP HyperText Transfer Protocol. 16, 17, 30, 43, 54

HTTPS HyperText Transfer Protocol Secure. 17, 43, 55

IT Information Technology. 87

JSON JavaScript Object Notation. 16, 57

JSX JavaScript XML. 48, 49, 50

JWT JSON Web Token. 16, 45, 56, 57

MADDPG Multi-Agent Deep Deterministic Policy Gradient. 46, 92, 93

MCTS Monte-Carlo Tree Search. 46, 91

MDP Markov Decision Process. 13

MitM Man-in-the-Middle. 15, 17, 43, 55

MLP Multi-Layer Perceptron. 10, 11, 46, 93

MSE Mean Squared Error. 11, 80, 91, 92, 106

NPM Node Package Manager. 27, 49, 50, 55

ORM Object-Relational Mapping. 27, 28, 44, 53

PPO Proximal Policy Optimisation. 2, 13, 15, 21, 22, 29, 46, 70, 75, 77, 82, 90, 91, 92, 94, 106

RL Reinforcement Learning. 1, 6, 10, 13, 15, 21, 22, 45, 52, 70, 94, 96, 97

SQL Structured Query Language. 15, 18, 44

SSL Secure Sockets Layer. 17

TD Temporal Difference. 14

TLS Transport Layer Security. 17

TSX TypeScript Execute. 59

UI User Interface. 26, 27, 34, 49

UK United Kingdom. 6, 8, 9, 26, 88

UML Unified Modeling Language. 33

URL Uniform Resource Locator. 30, 48, 49, 50, 51, 52, 54, 55, 56, 58, 60, 61, 63, 66

WC Water Closets. 10

WSGI Web Server Gateway Interface. 54

XSS Cross-Site Scripting. 15, 18, 44, 55

Chapter 1

Introduction

Real estate represents one of the leading sectors of human economic activity. It encompasses the buying, selling, developing, and management of land and properties. It includes residential spaces which house families, commercial properties that drive businesses, industrial sites that support manufacturing, and undeveloped land holding potential for future transformation.

Architecture represents another essential sector, defined as the art or practice of designing and constructing buildings. Its motivation reaches further than simply creating shelter, representing a fusion of human demands, cultural expression, technological innovation, and the natural world.

The intersection of architectural design, technological innovation, and artificial intelligence represents a transformative opportunity in spatial planning and real estate management. This report explores the development of a technological platform designed to address the multifaceted challenges seen in modern real estate management. It additionally envisions a system that can quickly generate and evaluate architectural floor plan design alternatives with precision and efficiency, that maximise spatial utility, minimise costs, and ensure full compliance with regulatory constraints.

1.1 Motivations

Traditional approaches to architectural design and property management are being greatly challenged. The conventional methodologies of spatial configuration - often reliant on static design principles, intuitive expertise, and manual optimisation - are proving inadequate in addressing the elaborate demands of contemporary real estate development.

The real estate landscape has undergone profound transformations in recent decades, evolving from a traditionally fragmented and manually intensive industry to an increasingly complex ecosystem demanding sophisticated management strategies. The total value of the world’s property stood at £293.4 trillion at the end of 2022, an 18.7% increase over the past three years [1], expecting to reach a value of £508.8 trillion by 2025 [2]. As global property markets become more interconnected and technologically driven, the need for intuitive, comprehensive management grows more critical. Amongst the biggest barriers to increased real estate activity were regulatory and compliance barriers (17%), limited expertise (8%) and access to capital or finance (7%) [3].

Architectural design continues to pose a sophisticated challenge that demands precise integration of functional requirements, economic constraints, and regulatory compliance. The exponential growth of urban populations, combined with increasingly rigid building regulations and rising costs of construction, necessitates a new conception of how spatial environments are implemented. In the United Kingdom (UK) alone, we see an increase from 21.2 million homes in 2001 to over 29.9 million homes currently [4]. The construction cost index (used to measure construction costs in the face of inflation, costs of supplies, energy prices, amongst other factors), with an index of 100 in the year 2000, increased rapidly to 223 in the first quarter of 2025 and is expected to reach over 240 in 2028 [5]. This represents over a 100% rise in construction costs over this time period.

Reinforcement Learning (RL) represents a transformative computational approach in addressing complex, multidimensional decision-making challenges across many domains. Whilst strongly demonstrated in areas such as game strategy, robotic control, and autonomous systems, its potential in architectural design remains largely unexplored. This research positions architectural space layout design as a sequential decision-making problem suited to algorithmic optimisation.

By integrating machine learning algorithms with architectural design principles, this project offers a new vision of spatial planning. The proposal is a system capable of analysing historical design data, offering a dynamic, adaptive approach to floor plan generation that can learn, evolve, and optimise across multiple design parameters simultaneously. It aims to transform a predominantly human-driven process to an adaptive, learning-based approach and create a tool that could benefit architects, urban planners, real estate developers, property managers, home owners, investors and many more.

1.2 Objectives

To counteract the complexity of real estate management (specifically the regulatory and compliance barriers, limited expertise and access to finance), the sophistication of architectural development along with the rising cost of construction, this project aims to construct a comprehensive, web application to aid in the management of real estate and provide an architectural tool capable of modifying existing floor plans with a focus on spatial optimisation, cost minimisation and legal adherence.

1.2.1 Real Estate Management

The proposal for the real estate management portion of this project is to provide functionality to retrieve actionable insights towards owned properties and facilitate universal management.

- Users should be able to create and manage portfolios, properties and rooms.
- Users should be able to track occupancy of properties and information on occupants.
- Users should receive information and statistical insights based on their properties, such as income, expenses, documents, loans, and prospective work.
- Users should receive useful information to aid management such as (local) laws and regulations, rent trajectories and property appreciation trajectories.

1.2.2 Architectural Optimisation

The proposal for architectural optimisation is to provide functionality to optimise existing floor plans to maximise space, minimise costs and adhere to legal compliance.

- Users should be able to recreate an existing floor plan.
- Users should receive an option of new optimised floor plans, based on their existing floor plan.
- Optimised floor plans must aim to maximise space, minimise costs and adhere to relevant legal compliance.
- Legal compliance should be inferred by the location of the property and its renting structure (if applicable).

Chapter 2

Background

Real estate management, architecture and reinforcement learning encompass many fundamental concepts.

2.1 Real Estate Legal Compliance

Within real estate development and management, there are numerous legal, regulatory requirements present across international jurisdictions. This project specifically examines the regulatory framework established in the UK for residential and commercial properties. The analysis centres on minimum habitability standards mandated for rental properties, encompassing crucial aspects of safety, living conditions, sanitation, and utility provisions (energy, water, electricity, and gas). The licensing requirements that govern residential letting in the UK are Selective Licensing and the Houses in Multiple Occupations (HMO) License, each imposing specific compliance criteria designed to safeguard tenant welfare and maintain property standards. This regulatory context provides essential parameters for the optimisation of floor plan design.

2.1.1 Selective Licensing

Selective licensing [6] is a regulatory framework introduced in the UK under the Housing Act 2004. It grants local authorities the power to establish licensing schemes for private rental properties within designated areas. These schemes typically target neighbourhoods experiencing significant anti-social behaviour, low housing demand, poor property conditions, or high migration. Under selective licensing, landlords must apply for a license for each rental property

they own within the designated area. They must demonstrate that they are "fit and proper" persons and that their properties meet essential safety and management standards. The license application process typically involves background checks, property inspections, and compliance with various housing regulations including gas safety, electrical safety, and energy performance standards. Licenses generally remain valid for five years and incur fees that vary by local authority. This system aims to improve housing standards, combat rogue landlords, and enhance neighbourhood conditions by ensuring professional management of private rental properties while giving councils enforcement powers against non-compliant landlords through fines and prosecution.

2.1.2 Houses in Multiple Occupations (HMO)

Houses in Multiple Occupations (HMO) licensing [7] in the UK is a regulatory framework established under the Housing Act 2004 that applies to properties occupied by three or more tenants from different households who share facilities such as bathrooms and kitchens. It operates on two tiers: mandatory licensing for larger HMOs (for those with five or more occupants or two or more separate households) and additional licensing schemes that local authorities can implement to cover smaller HMOs within their jurisdiction. To obtain an HMO license, landlords must demonstrate compliance with rigorous safety standards. This includes fire safety precautions, adequate escape exits, sufficient amenities for the number of occupants, and proper waste management systems. The application process requires detailed property information, floor plans, safety certificates, and proof that the landlord is a "fit and proper person". Licenses typically last for five years, with fees varying by local authority, and carry specific conditions regarding maximum occupancy and management requirements. Non-compliance can result in substantial financial penalties, rent repayment orders, or prosecution, aiming to safeguard vulnerable tenants and maintaining adequate housing standards in shared accommodation.

This license specifies the following mandatory, minimum sleeping room sizes:

- 6.51 squared metres for one person over 10 years of age.
- 10.22 squared metres for two persons over 10 years.
- 4.64 squared metres for one child under the age of 10 years.

The mandatory room size conditions mentioned above present a statutory minimum and do not represent the optimal room size. Local housing authorities have discretion to demand

higher standards within licence conditions, but must not set lower standards. These sizes are in addition to any en-suites or Water Closets (WC).

Whilst not outlined in the mandatory minimum conditions by GOV.UK, there are many local housing authorities which also specify minimum kitchen sizes, shared area sizes and studio sizes. The typical minimum kitchen size requirement is at least 7 squared metres.

2.2 Reinforcement Learning (RL)

Reinforcement Learning (RL) is a learning paradigm in which an agent interacts with an environment to achieve a goal, such as winning a game or designing a layout.

The agent learns by:

1. Observing the current state of the environment.
2. Processing the observations using a chosen algorithm to decide on an action.
3. Taking an action, which changes the environment's state.
4. Receiving a reward as feedback.

By maximising cumulative reward, the agent learns a policy - a strategy for selecting the best actions in any given state.

2.2.1 Artificial Neural Network (ANN) and Multi-Layer Perceptron (MLP)

An Artificial Neural Network (ANN) [8] is a machine learning algorithm, that aims to make decisions in a similar manner to the human brain. Its design revolves around mimicking biological neurons as they work together to identify phenomena, weigh options and arrive at conclusions. An ANN consists of multiple layers of artificial neurons or nodes. This includes an input layer, one or more hidden layers, and an output layer. Each node is connected to others, and carries an associated weight and threshold. If the output of any node is above a specified threshold value, that node is activated and sends data to the next layer of the network. ANNs use training data to improve their accuracy over time and essentially learn. A testing or validation set is used, distinct from the training set to quantify the performance of the network.

Each node functions as its own linear regression model composed of input data, weights, a bias or threshold, and an output. Once an input layer is configured, weights are assigned

to the nodes. Weights determine the importance of any given variable, with a larger weight contributing more significantly to the output. Inputs are multiplied by their respective weights and summed. This value is then passed through an activation function, which determines the output. If this output exceeds a given threshold, it activates the node, passing data to the next layer in the network. The output of one node then becomes the input of the next node. This passing of data through layers defines this neural network as a "feedforward" network.

The activation function can be specified as:

$$activation = b + \sum_{i=1}^n w_i x_i$$

where b is the bias, n is the number of inputs from the incoming layer, i is a counter from 1 to n , w is the weight and x is the input to the neuron.

This represents the sum of all weighted inputs plus a threshold value. The output can be defined as 1 if the activation function produces a value equal to or greater than 0 and 0 if the activation function produces a value less than 0.

As we train an ANN model in this way, we evaluate its accuracy using a cost (or loss) function. This is also commonly referred to as the Mean Squared Error (MSE). Ultimately, the goal is to minimise the cost function to ensure correctness of fit for a given observation. As the model adjusts its weights and bias, it uses the cost function and reinforcement learning to converge (or terminate). Weights are adjusted through gradient descent, allowing a model to determine the direction to take to minimise the cost function, consequently reducing errors. With each training example, the parameters of the model adjust to gradually converge at the minimum.

The cost function is

$$CostFunction = MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

where n is the number of samples or data points, i is a counter from 1 to n , y_i is the observed value and $\hat{y}_i$ is the predicted value.

A Multi-Layer Perceptron (MLP) is a versatile type of ANN that consists of multiple lay-

ers of fully connected neurons, enabling the approximation of complex non-linear relationships between inputs and outputs.

2.2.2 Linear Regression

Linear regression analysis is used to predict the value of a variable based on the value of another variable. The variable being predicted is referred to as the dependent variable. The variable used to predict the dependent variable's value is the independent variable. Linear regression provides a linear relationship between the dependent variable and the independent variable to predict future outcomes.

The linear relationship between these two variables (the line of regression) is calculated by:

$$Y = a + bX + u$$

where Y is the dependent variable being predicted, a is the y-intercept (where the line intercepts the y-axis on a graph), X is the independent variable being used to predict Y and u is the error term.

2.2.3 Gradient Descent

Gradient descent is an optimisation algorithm used to train machine learning models by minimising errors between predicted and actual results. It starts with an arbitrary point used to evaluate performance. From this point, the derivative (or slope) is found. A tangent line is used to observe the steepness of the slope. The slope will inform the updates to the weights and bias. The slope at the starting point will be steeper, but as new parameters are generated, the steepness should gradually reduce until it reaches the lowest point on the curve, known as the point of convergence.

This is done by applying

$$w_i \leftarrow w_i - \alpha \frac{\partial}{\partial w_i} Loss(w)$$

until convergence, where α is the learning rate and controls how big a step we take towards the lowest point, $\alpha \frac{\partial}{\partial w_i}$ is the gradient and $Loss(w)$ represents a cost or loss function.

2.2.4 Markov Decision Process (MDP)

A Markov Decision Process (MDP) is one of the most effective models for framing an RL problem. It ensures that all the information an agent needs to make a decision is embedded in its current state.

The MDP framework consists of four key components:

1. The Observation Space - What the agent perceives at each step.
2. The Action Space - The set of actions the agent can take.
3. A Transition Model - A function that specifies how an environment changes after an action has been taken.
4. The Reward Function — The feedback provided by the environment that guides the agent's learning.

By defining floor plan design optimisation as an MDP, we enable an RL agent to iteratively refine space layouts through interaction and learning.

2.2.5 Proximal Policy Optimisation (PPO)

Proximal Policy Optimisation (PPO) is a reinforcement learning algorithm that improves upon previous policy gradient methods by limiting policy updates to prevent large, destabilising changes. It typically uses two neural networks:

- A policy network - that outputs a probability distribution over actions given a state $(\pi_{\theta}(a|s))$.
- A value network - that estimates the expected return from a state.

It works as follows:

1. Collect a batch of experiences using the current policy.
2. Compute advantages and returns for each time step.
3. Perform multiple epochs (a complete pass through the collected batch of experiences) of optimisation on the collected data.
4. Update the policy parameters using the clipped objective.
5. Update the value function to better predict returns.

6. Repeat.

Advantages are computed for each time step using Generalised Advantage Estimation (GAE). This is done by:

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda)} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}$$

where $\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$ is the Temporal Difference (TD) residual (the difference between the observed reward plus the discounted value of the next state and the current value estimate), γ is the discount factor and λ is the GAE parameter.

The clipping mechanism ensures that the policy update stays within a "trust region" around the old policy, providing more stable learning than methods that might make dramatic policy changes. This works by applying:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}[\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)]$$

$r_t(\theta)$ represents the probability ratio between new and old policies, A_t is the advantage estimate, and ϵ is the clipping hyperparameter.

The ratio function $r_t(\theta)$ calculates the probability of taking action a_t at state s_t in the current policy divided by the previous one:

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$$

The value function update aims to minimise the value function loss to improve return predictions. This is done by:

$$L^{\text{VF}}(\phi) = \mathbb{E}_t \left[\left(V_\phi(s_t) - \hat{R}_t \right)^2 \right]$$

where $\hat{R}_t$ is the empirical return (the cumulative discounted reward collected from time step t).

Optionally, an entropy bonus can be added to encourage exploration:

$$L^{\text{TOTAL}}(\theta) = L^{\text{CLIP}}(\theta) - c_1 L^{\text{VF}}(\phi) + c_2 \mathbb{E}_t [S[\pi_\theta](s_t)]$$

where S is the entropy bonus term. Exploration refers to the agent attempting new actions rather than exploiting actions that they have learnt. This helps the agent ascertain a better understanding of the environment, learn new optimal actions and therefore learn to optimise better.

To generate optimal space layouts, a RL agent can be trained using Proximal Policy Optimisation (PPO). The agent can interact with the environment to find a design solution for the given design scenario. A layout would be considered a good solution when its geometrical and topological properties closely match those defined in the input design scenario.

2.2.6 Exploration and Exploitation

The exploration-exploitation tradeoff is a key underlying concept in machine learning, particularly in Reinforcement Learning (RL) and decision-making systems. It describes the balance between using current knowledge to maximise immediate rewards (exploitation) and gathering new information that might lead to better rewards in the future (exploration). Strict exploitation can risk an agent getting stuck in a suboptimal solution, missing potentially better strategies and ultimately performing poorly in a dynamic environment. Excessive exploration can lead to an inefficient use of resources, slow convergence to good solutions and lower short-term performance. Balancing the two is an extremely environment-specific task and can depend on factors such as available resources (such as time and computation capacity), how dynamic an environment is, the cost of certain decisions and ultimately whether the optimisation task is a long term or a short term goal.

2.3 Security

A cyber attack is an offensive, unauthorised system/network access by a third party. It aims at destroying or stealing confidential information from a computer network, information system, or personal device. The person who carries out this cyber attack is called a hacker [9]. As a web application, this project can be especially prone to cyber attacks such as Man-in-the-Middle (MitM) attacks, Denial-of-Service (DOS) attacks, SQL Injection, and Cross-Site Scripting (XSS) attacks.

2.3.1 Sessions

A session refers to a user's time browsing a web site. A web session is a series of contiguous actions by a visitor on an individual website within a given time frame. To track sessions, a web session ID is stored in a user's browser. This session ID is passed along with any HyperText Transfer Protocol (HTTP) requests that the visitor makes while on the site [10]. This allows a server identify a client requesting access to a service, providing a means of authentication. It can also be used to personalise user experience, to store data temporarily in the browser and to track user behaviour [11].

2.3.2 JSON Web Token (JWT)

JSON is a standard text-based format for representing structured data based on JavaScript object syntax [12]. A JSON Web Token (JWT) is a standard for securely transmitting information between parties as a JSON object. This information can be verified and trusted because it is digitally signed. When a user logs in, the browser sends the login credentials to the server. The server generates a JWT if the credentials are valid containing user data, signed with a secret key. The JWT is then sent back to the client and stored in local storage. When a user attempts to access a protected route, i.e. a page that requires authentication, the client includes the JWT in the request which the server verifies and responds to [13].

JWTs provide a fast, secure way of authenticating users as they traverse through different web pages. They are small in size and so can be passed between two entities quickly, are easy to generate and allows you to specifically control what a user has access to, how long the permission lasts and what a user can do whilst logged in. However, JWTs use a single key, meaning if compromised, the entire system is at risk. They are also complex to understand with heavy use of cryptographic signature algorithms. If used incorrectly, it can be a potential risk [14].

Access Token

An access token is like a digital key, often in the form of a JWT, that lets users access resources without needing to log in repeatedly. When a user wants to access resources through a client application, the user gives permission for the application to get an access token from an authorisation server. The authorisation server sends an access token to the application. The application checks the token and if it is valid, it allows the user to access the resources. Access tokens usually have a very short lifespan, typically a few hours [15].

Refresh Token

A refresh token extends the life of an access token. When an access token expires, a refresh token can be used to get a new one without making the user log in again. Refresh tokens are usually stored securely on the authorisation server. They work together with access tokens to allow longer sessions, improving user experience and security. Unlike access tokens, refresh tokens last much longer, up to 90 days in some systems. Refresh tokens make managing sessions easier, convenient and more secure [15].

2.3.3 HTTP and HTTPS

HyperText Transfer Protocol (HTTP) is a protocol used for transferring data over a network. A protocol is a syntax for presenting information. Most information that is sent over the Internet, including website content and API calls, uses the HTTP protocol. HTTP requests are generated by a user's browser as the user interacts with web properties.

HyperText Transfer Protocol Secure (HTTPS) uses Transport Layer Security (TLS) or Secure Sockets Layer (SSL) to encrypt HTTP requests and responses, thus making them secure. TLS and SSL are security public key encryption protocols designed to facilitate privacy and data security for communications over the Internet [16] [17]. When a client opens a connection with a server, the two devices use the public and private key to agree on new keys, called session keys, to encrypt further communications between them. All HTTP requests and responses are then encrypted with these session keys, so that anyone who intercepts communications can only see a random string of characters, not the plaintext [18].

2.3.4 Man-in-the-Middle (MitM) Attacks

In a MitM attack, an attacker intercepts and manipulates communication between two parties without their knowledge, for example, session hijacking - stealing active user sessions to gain unauthorised access [9].

2.3.5 Denial-of-Service (DOS) Attacks

A Denial-of-Service (DOS) attack aims to overwhelm a system, server, or network by flooding it with excessive requests, causing slowdowns or crashes [9].

2.3.6 SQL Injection

SQL injection is a code injection attack in which attackers insert malicious SQL queries into the input fields of web applications to manipulate databases. This allows hackers to retrieve, modify, or delete sensitive data, such as user credentials or personal information [9].

2.3.7 Cross-Site Scripting (XSS) Attacks

XSS attacks occur when an attacker injects malicious scripts into a legitimate website, which executes in a user's browser [9].

2.3.8 Cross-Site Request Forgery (CSRF) Attacks

Cross-Site Request Forgery (CSRF) Attacks are a class of attack where unauthorised commands are transmitted to a website from a trusted user. Because they inherit the user's session information, they appear to be valid commands. This allows a hacker to manipulate user behaviour.

2.3.9 Cross-Origin Resource Sharing (CORS)

The same-origin policy is a browser security feature that restricts how documents and scripts on one origin can interact with resources on another origin. This prevents a script being compromised by an attacker [19]. A browser's same-origin policy blocks reading a resource from a different origin. This mechanism stops malicious sites from reading other sites' data, but it also prevents legitimate uses.

Cross-Origin Resource Sharing (CORS) is a browser mechanism which enables controlled access to resources located outside of a given domain. Enabling CORS lets the server tell the browser it can use an additional origin. CORS vulnerabilities can arise primarily due to misconfigurations [20]. This requires careful configuration.

Chapter 3

Literature Review

3.1 ArchiGAN

This study [21] explores using a Generative Adversarial Network (GAN) to analyse and generate floor plans. A GAN works in two parts. The generator learns to generate plausible data. The generated instances become negative training examples for the discriminator. The discriminator learns to distinguish the generator's fake data from real data. The discriminator penalises the generator for producing implausible results. When training begins, the generator produces obviously fake data, and the discriminator quickly learns to tell that it's fake. Using a GAN model enables capturing more complexity across floor plans and breaking down the complexity of the problem through successive steps. Each step corresponds to a model, specifically trained for its task. These steps successively come together to generate floor plans.

The study outlines its goals as optimising the generation of a large and highly diverse quantity of floor plan designs, to offer a proper classification methodology to qualify floor plans and to allow users to browse through generated design options. It focuses on six key metrics to qualify, optimise and generate floor plan designs. These are footprint shape, orientation, thickness & texture, program, connectivity, and circulation. The footprint shape refers to the boundary of the floor plan. The orientation refers to where walls face within a floor plan. The thickness & texture refer to the size and feel of the walls within a floor plan. The program refers to the type of rooms that a floor plan consists of. The connectivity relates to the adjacency of the rooms, i.e. which rooms are next to each other and how far they are from one another. The circulation captures how people move within a floor plan.

By training separate models in this way and using them in conjunction to generate floor

plans, the ArchiGAN application is able to monitor several parameters crucial in architecture design and account for all of them simultaneously.

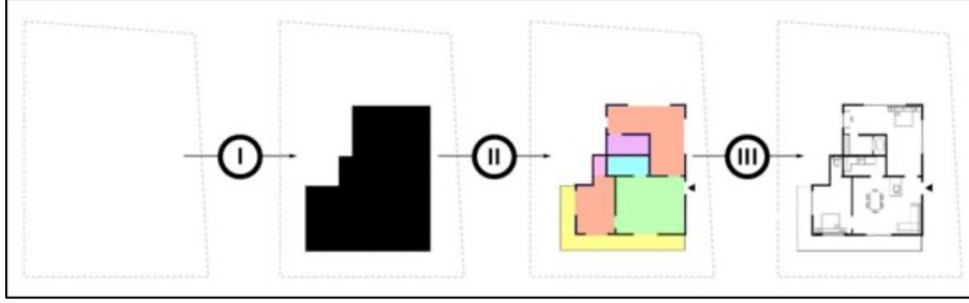


Figure 3.1: ArchiGAN Three-Step Generation Stack

Another fundamental aspect of the generation of floor plans in this study is the style and organisation. To investigate architectural style learning, they trained and tuned an array of models on specific styles - Baroque, Row House, Victorian Suburban House, & Manhattan Unit. In this way, the models were able to emulate each particular architectural style.

A significant contribution of this work is its user-centric interface design, which enables efficient creation of building footprints, primary entrances, and window placements. The system then autonomously completes the design with appropriate room divisions, additional windows, entrances, and furniture arrangements.

Whilst this study offers significant insights into the essential functionality of floor plan spaces, such as the layout, culture and connectivity, it heavily overlooks critical aspects such as space efficiency, architectural legal compliance and cost. Consequently, whilst beneficial in expressing culture through floor plan design and perhaps focused more towards residential properties, it falls short in maximising internal value of a property and maximising revenue of commercial properties. This study also doesn't consider the modification of existing floor plans but instead focuses on generating a large number of new designs from scratch. Whilst this is beneficial for its task, and would benefit real estate owners and investors who are looking to build new properties, it wouldn't accommodate managers with existing properties who are aiming to optimise an existing floor plan design. This could mean floor plan spaces would have to be constructed from scratch and would not benefit from finding the minimal amount of changes to achieve maximal space, which would minimise costs and ultimately maximise revenue.

A potential improvement stated in this study is considering the position of load-bearing

walls and columns as inputs to allow a more conformative placement of rooms and walls as per the original building structure. This is a vital consideration in floor plan design, since some walls may not be able to move so as to not compromise the structural integrity of the building. This can prove to be a beneficial aspect to consider in this project.

3.2 SpaceLayoutGym

SpaceLayoutGym [22] presents an alternative perspective on spatial optimisation using Proximal Policy Optimisation (PPO). It introduces a "laser-wall" partitioning technique that bridges vector-based and pixel-based methods, where walls emit imaginary light beams to divide spaces and provides two planning strategies. These are one-shot planning (generating layouts in a single pass) and dynamic planning (allowing adaptive refinement).

The agent learns through episodic interaction with the SpaceLayoutGym environment. Each episode in dynamic planning begins with an initial random wall configuration and terminates when a satisfactory layout is achieved or a maximum number of transformations is reached. The agent gradually improves its policy by maximising cumulative rewards across episodes. This allows the system to learn design strategies that balance local optimisations (individual room properties) with global constraints (overall layout coherence and adjacency requirements).

The research demonstrates RL agents' success in generating architectural layouts that balance geometric constraints (room areas, aspect ratios) and topological requirements (adjacencies between rooms, access to facades). The method shows particular strength in its ability to create diverse, functional designs that intuitively mimic human design processes.

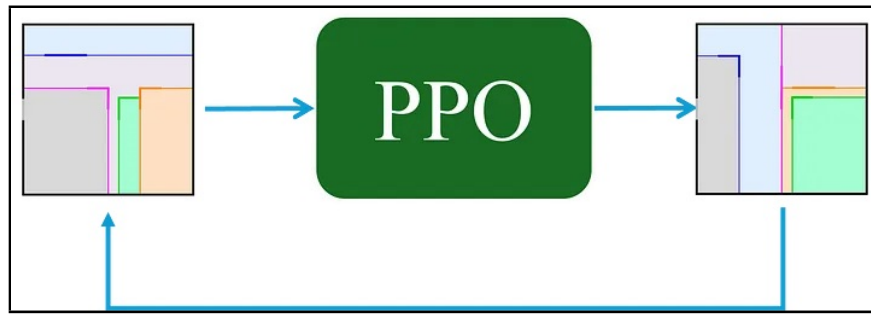


Figure 3.2: SpaceLayoutGym Dynamic Planning Process

This research frames the layout design as a sequential decision-making process, balancing space utilisation and design constraints. It features an innovative way of creating walls and

determining movement within a floor plan space, achieving high precision in meeting design requirements and a strong capacity for balancing multiple objectives simultaneously. Unlike ArchiGAN, it does not focus on learnt movement within spaces but instead on adjacencies to a common room. It also experiments with angled versus straight walls. Whilst this study offers significant insights into achieving space efficiency within a floor plan design, it does not consider any legal compliance framework or cost minimisation methods. It similarly focuses on generating floor plans from scratch and not on modification.

One common theme between these two studies is the challenge of balancing multiple design objectives, such as geometric constraints, topological relationships, and aesthetic considerations within floor plan design. Both papers explore methods for generating diverse and functional layouts while adhering to these constraints. They also address the need for efficient exploration space inherent in floor plan design.

While both papers aim to automate floor plan design optimisation, they use fundamentally different methodologies, with Reinforcement Learning (RL) offering a procedural, refinement-based approach and Generative Adversarial Network (GAN) providing a generative, data-driven alternative. Although GANs demonstrate immense value in an architectural environment, it pivots around the idea of generation. PPO, however, centres on direct optimisation through immediate interaction with an environment. PPO also provides for reward-based learning which allows it to balance distinct constraints simultaneously, unlike GANs which would have to consider each constraint separately and hence may struggle to find an effective equilibrium. The sequential methodology behind PPO allows for iterative feedback and exploration of optimal solutions.

Chapter 4

Requirements & Specification

The requirements and specification regarding the two portions of real estate management and architectural optimisation in a web application are as follows.

4.1 Requirements

These requirements are sectioned between the functional and non-functional requirements of real estate management and architectural optimisation.

4.1.1 Functional Requirements

Web Application:

- **FR1** - The system must facilitate secure sign up for an account.
- **FR2** - The system must facilitate secure logging in to an account.
- **FR3** - The system must facilitate secure logging out of an account.
- **FR4** - The system must facilitate secure account profile editing.

Real Estate Management:

- **FR5** - The system must facilitate creating a new portfolio of properties.
- **FR6** - The system must facilitate viewing a portfolio of properties.
- **FR7** - The system must facilitate deleting a portfolio of properties.

- **FR8** - The system must facilitate adding a property to a portfolio.
- **FR9** - The system must facilitate adding information about a property in a portfolio (total rent, bills, occupancy, costs, loans, licenses, documents, prospective works).
- **FR10** - The system must facilitate viewing relevant information about a property in a portfolio.
- **FR11** - The system must facilitate editing a property in a portfolio.
- **FR12** - The system must facilitate deleting a property from a portfolio.
- **FR13** - The system must facilitate adding a room to a property.
- **FR14** - The system must facilitate adding information about a room in a property (rent, bills, occupancy, costs, tenants, documents, prospective works).
- **FR15** - The system must facilitate viewing relevant information about a room in a property.
- **FR16** - The system must facilitate editing a room in a property.
- **FR17** - The system must facilitate deleting a room from a property.
- **FR18** - The system must display an analysis of cash flow for all portfolios and properties as a dashboard.
- **FR19** - The system must only allow the intended user to view, edit, delete portfolios, properties, rooms and respective information.
- **FR20** - The system must facilitate easy access to information to aid management (licenses, laws, mortgages).

Architectural Optimisation:

- **FR21** - The system must facilitate creating a new floor plan.
- **FR22** - The system must facilitate editing an existing floor plan.
- **FR23** - The system must be able to read an existing floor plan.

- **FR24** - The system must be able to identify relevant legal compliance and regulations.
- **FR25** - The system must be able to optimise an existing floor plan to maximise space, minimise costs and adhere to relevant legal compliance and regulations.
- **FR26** - The system must be able to provide multiple, distinct optimisation options.

Database Operations:

- **FR27** - The system must store new account details in the database.
- **FR28** - The system must verify user log in with validated details from the database.
- **FR29** - The system must store edited account details in the database.
- **FR30** - The system must store new portfolio details in the database.
- **FR31** - The system must store edited portfolio details in the database.
- **FR32** - The system must remove portfolio details in the database if a portfolio has been deleted.
- **FR33** - The system must store new property details in the database.
- **FR34** - The system must store edited property details in the database.
- **FR35** - The system must remove property details in the database if a portfolio has been deleted.
- **FR36** - The system must store new room details in the database.
- **FR37** - The system must store edited room details in the database.
- **FR38** - The system must remove room details in the database if a portfolio has been deleted.
- **FR39** - The system must store new floor plan details in the database.
- **FR40** - The system must store edited floor plan details in the database.
- **FR41** - The system must remove floor plan details in the database if a portfolio has been deleted.

4.1.2 Non-Functional Requirements

- **NFR1** - The system must be easy to understand.
- **NFR2** - The system must be easy to use.
- **NFR3** - The system must comply with UK data regulations.

4.2 Specification

This section presents a comprehensive analysis of the software technologies selected for this project.

4.2.1 Frontend Architecture

The frontend architecture comprises the client-side components that users directly interact with through their browser. After evaluating several potential frameworks, a Vite-powered React implementation was selected as the optimal solution.

React

React is a JavaScript library for creating user interfaces. React was selected after evaluating Angular, Vue, and Svelte. Angular offers a comprehensive framework but introduces excessive complexity for this application. Vue provides simplicity but lacks React’s extensive ecosystem for spatial visualisation libraries. Svelte delivers excellent performance but presents integration challenges with specialised geospatial libraries required for floor plan rendering.

The component-based architecture perfectly suits modular requirements, allowing UI elements to be created and reused throughout the application, streamlining the development process and enhancing maintainability. It incorporates a virtual Document Object Model (DOM) (a data representation of the objects that comprise the structure and content of a document on the web) implementation which optimises rendering of complex layouts. The unidirectional data flow of React additionally simplifies state management [23].

React’s architecture directly supports the sophisticated UI requirements for visualising before/after floor plans and providing interactive controls for the reinforcement learning optimisation process as well as functionality for real estate management. The component-based architecture enables separation of concerns between the different functional areas of the application (real estate management and floor plan optimisation).

Vite

Vite is a build tool that facilitates the configuration of a development environment for React frameworks. Vite was chosen over alternatives like Create React App (CRA) and Webpack due to its superior performance characteristics and developer experience benefits. While CRA offers simplicity, it suffers from increasingly slow rebuild times as applications grow. Webpack provides extensive customisation but requires complex configuration.

Vite leverages native ECMAScript (ES) modules (a standardised way to organise and share JavaScript code, allowing developers to split their code into separate files) for near-instantaneous hot module replacement (a way of exchanging modules in a running application), delivering 10-100x faster development server startup times compared to webpack-based alternatives. It employs highly optimised production builds through Rollup.js and integrates seamlessly with Node Package Manager (NPM), the default package manager for the JavaScript runtime environment Node.js. It additionally provides out-of-the-box TypeScript support for improved code reliability [24].

The selection of Vite directly addresses one of the project’s core requirements: rapid development iterations to refine the UI for floor plan visualisation and manipulation. This efficiency is crucial when iteratively testing UI elements that will display complex spatial optimisation results.

4.2.2 Backend Architecture

The backend architecture consists of server-side components responsible for data processing, reinforcement learning, and core business logic. A Python-based stack was selected to leverage its extensive machine learning ecosystem while providing robust web application capabilities.

Django

Django is a high-level Python web framework that enables rapid development of secure and maintainable websites. Django was selected after evaluating Flask, FastAPI, and Express.js alternatives. While Flask offers simplicity, it lacks Django’s built-in features critical for user management and security. FastAPI provides performance advantages but lacks Django’s mature ecosystem for spatial data handling. Express.js offers JavaScript consistency but would create language barriers with the Python-based reinforcement learning components.

Django includes a built-in authentication system which accommodates multi-user access with varying permission levels, Object-Relational Mapping (ORM) which simplifies storage and

retrieval of data, an administrative interface to enable rapid debugging and data management during development, robust security features and a templating system which facilitates server-side rendering beneficial for performance [25].

Django’s philosophy directly addresses the need for rapid development of the web application while maintaining security and scalability. Its ORM capabilities are particularly valuable for modelling complex relationships between real estate portfolios, properties, floor plans and the optimisation process.

Gymnasium

Gymnasium is an open-source Python library that provides a standardised Application Programming Interface (API) for reinforcement learning environments. It offers a collection of environments where reinforcement learning agents can be trained and tested. Gymnasium was selected over alternatives like PettingZoo and custom environment implementations. PettingZoo focuses on multi-agent scenarios, which exceed current project requirements. Custom environments would require significant development effort to replicate Gymnasium’s well-tested infrastructure.

Gymnasium provides a standardised environment interface ensuring compatibility with multiple reinforcement learning algorithms. It includes built-in wrappers to facilitate observation and action space transformations for data, allows for vectorised environments which enable parallel training to accelerate optimisation discovery, contains robust monitoring tools track agent performance across optimisation metrics and has integration with visualisation tools to simplify debugging of spatial reasoning processes [26].

Gymnasium provides the critical foundation for translating floor plan optimisation problems into a format suitable for reinforcement learning and implementing algorithms to ensure consistent optimisation. Its standardised interface allows for future algorithm experimentation without architectural changes.

Stable-Baselines3

Stable-Baselines3 is a set of reliable implementations of reinforcement learning algorithms in PyTorch. Stable-Baselines3 was chosen after evaluating TensorFlow Agents, RLlib, and custom implementation approaches. TensorFlow Agents lacks PyTorch integration, which was preferred for its dynamic computation graph. RLlib offers distributed training but introduces excessive complexity for initial development. Custom implementations would require substan-

tial development time without guaranteed stability.

Stable-Baselines3 contains a production-ready implementation of the PPO algorithm. PyTorch integration enables dynamic computational graphs for complex spatial reasoning. The modular design of Stable-Baselines3 allows future replacement of PPO with alternative algorithms if needed. It includes comprehensive logging which facilitates tracking of optimisation metrics during training and serialisation capabilities which enable saving and loading trained models for deployment [27].

The selection of Stable-Baselines3 directly addresses the requirement for a reliable, well-tested reinforcement learning implementation that can be applied to the domain of floor plan optimisation without excessive algorithm development effort.

Matplotlib

Matplotlib is a comprehensive library for creating static, animated, and interactive visualisations in Python. Matplotlib was selected after evaluating alternatives including Plotly, Seaborn, and D3.js. Plotly offers interactive visualizations but with higher overhead. Seaborn provides statistical visualization but lacks the low-level control needed for floor plans. D3.js offers powerful web-based visualization but presents integration challenges with the Python backend.

Matplotlib allows for comprehensive control over visual elements essential for accurate floor plan rendering and has seamless integration with NumPy (a package for scientific computing in Python) arrays used in spatial calculations. It has the ability to generate both static images for reports and interactive figures for development [28].

Matplotlib enables clear visualisation of the reinforcement learning process and results, a critical requirement for demonstrating the effectiveness of the spatial optimisation algorithms and communicating outcomes to non-technical stakeholders.

Shapely

Shapely is a Python package for manipulation and analysis of planar geometric objects. It is built on top of the Geometry Engine, Open Source (GEOS) library and provides a comprehensive set of geometric operations. Shapely was selected after evaluating GeoPandas, PyGeometry, and custom geometric implementation approaches. GeoPandas adds unnecessary Geographic Information System (GIS) functionality with increased overhead. PyGeometry lacks the comprehensive operations needed for complex floor plan analysis. Custom implementations would require extensive development and testing to match Shapely’s proven reliability.

Shapely includes robust implementation of fundamental geometric operations (intersections, unions, differences), efficient algorithms for spatial queries essential for constraint checking, seamless integration with Matplotlib for visualisation of processing results, validation capabilities ensure geometric integrity of shapes and serialisation support for storing and retrieving complex geometries [29].

Shapely is essential for processing and analysing floor plan geometries. It enables complex spatial operations such as determining room areas, calculating adjacency between spaces, checking for overlaps, and ensuring that modified floor plans maintain proper geometric relationships. These capabilities are crucial for implementing constraints related to legal compliance and practical space usage in the reinforcement learning environment. Shapely’s efficient geometric algorithms make it possible to quickly evaluate many potential floor plan modifications during the training and optimisation process.

4.2.3 Connecting the Backend and the Frontend

The backend and frontend are essentially two distinct systems that work together to provide the overall application. To use these in conjunction, we need an API to connect the two systems.

Django REST Framework

Django REST Framework acts as a wrapper over the default Django Framework. The REST framework facilitates serialisation (converting a model’s data to JSON/XML format), rendering this data to the view and creating URLs for mapping to the views [30]. This allows us to create and send valid data to the frontend.

Axios

Axios is a JavaScript library that communicates with servers on the internet to make HTTP requests. It can request data from a server, send data to the server and receive data back [31]. This allows the Vite frontend to send and request data such as models, views and URLs from the Django backend, correctly formatted by the Django REST Framework, via Axios and receive data back.

Chapter 5

Design

This section presents a high level design of the system, based on the background, requirements and specification.

5.1 System Architecture

The system architecture of this project consists of a data management layer, containing the database which stores the data used by the application, an application layer, containing the logic within the application and a presentation layer, containing the features that are presented to the user.

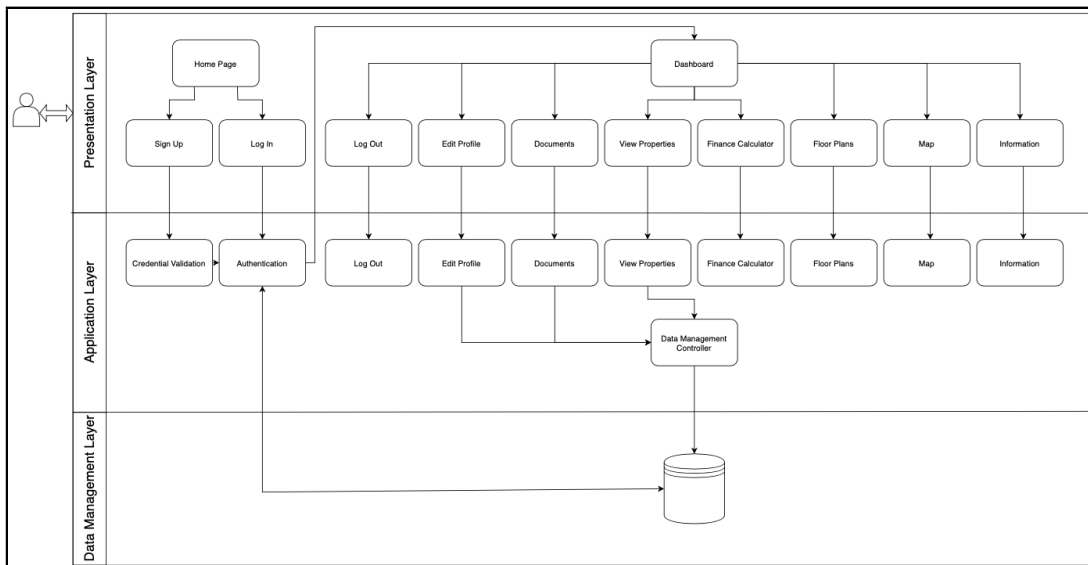


Figure 5.1: System Architecture Diagram

5.2 Use Cases

A use case describes a specific scenario where a user interacts with a system to achieve a particular goal.



Figure 5.2: Use Case Diagram

This use case diagram represents all the ways a user can interact with the system. The «include» tag represents a use case that is mandatory and part of the base use case. The «extend» tag represents a use case that is optional and can come after the base case.

5.3 System Structure & Data Management

Structural Unified Modeling Language (UML) diagrams show how the system is structured, including the classes, objects, packages, components, etc. in the system and the relationships between those elements [32].

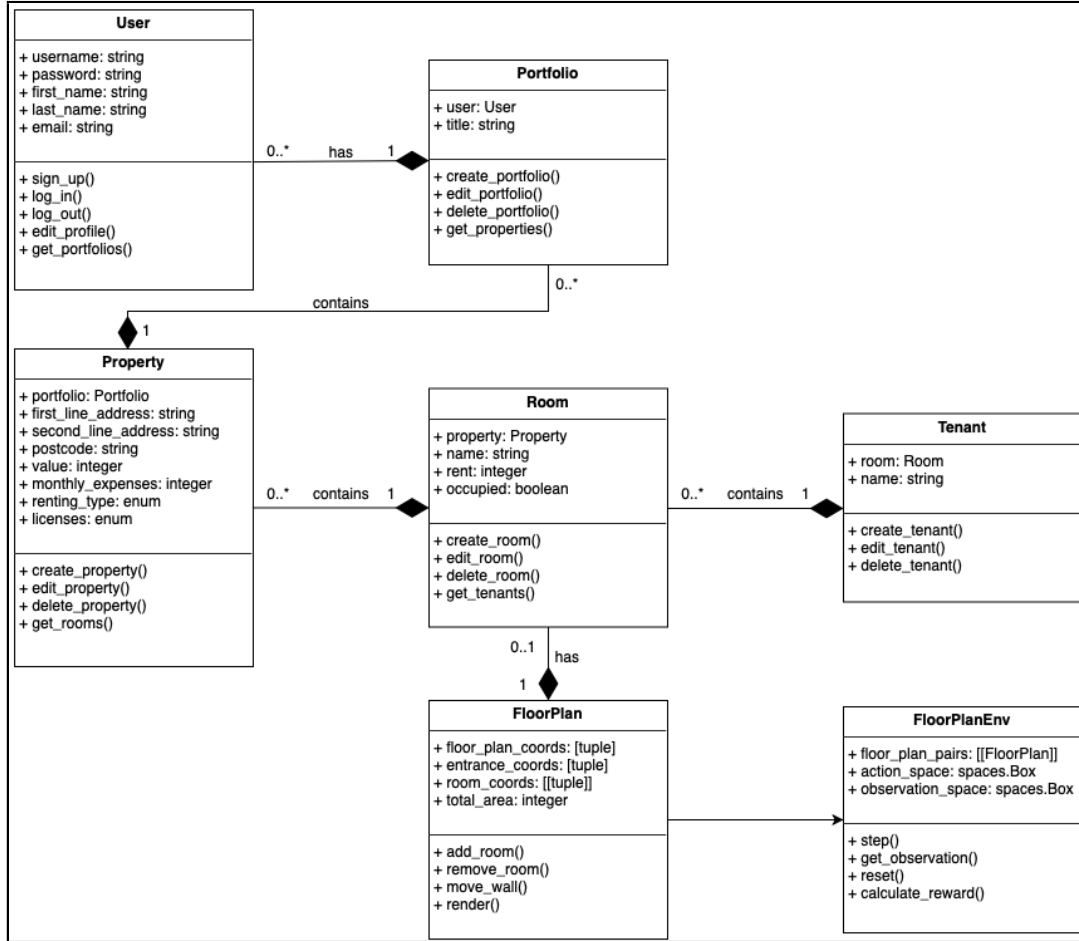


Figure 5.3: UML Diagram

This diagram defines classes for a user, portfolio, property, room, tenant, floor plan and an environment for floor plan optimisation. It defines the methods for each class and outlines the multiplicity of each relationship. The classes defined are representative of how the data is stored within the database. A database allows us to store data that will be used in the web application. In choosing the best database for this project, I've evaluated PostgreSQL, Oracle, MongoDB and SQLite.

PostgreSQL demonstrates high reliability, stability and an ability to handle a large amount

of data without crashing or losing data. It provides a high degree of flexibility, making use of complex data structures and is also highly scalable, allowing it to handle a large amount of data and concurrent users. However, PostgreSQL comes with a great level of complexity, especially to set up and maintain.

We also see high reliability and scalability with Oracle with the addition of excellent performance, known for its ability to handle large amounts of data and complex queries quickly and efficiently. This is also met with a high level of complexity and requires a high cost to use [33].

MongoDB [34] also presents high scalability and performance, with easy maintenance and support for complex data structures. However, it has increased memory usage and high resource consumption in comparison to other database alternatives .

SQLite [35] was the selected option for this project. Although SQLite has limited scalability, limited support for complex data types and limited concurrency in comparison to the database alternatives mentioned above, SQLite is the default database configured within Django and therefore requires no additional set up. Since the data needed to be stored is of minimal complexity, the ease of use, the embeddability, high reliability and cross-platform functionality outweighs the need for complexity shown by other database systems. SQLite is also very lightweight with a small library size and serverless, meaning there is no need to set up and maintain a separate database server process. This makes it easy to deploy and manage, as there are no additional dependencies.

Since SQLite is well integrated into Django, using this database software presents the ideal option for data management. This integration is seen in the form of Django models which represent Python classes. These map to the tables in the database. This allows for efficient handling of data.

5.4 User Interface

This section presents provisional designs for the UI for the different web pages of the application.

5.4.1 Home, Login and Sign Up

Home

The home page is the screen the user first sees when loading the application. It should present some information about the application and provide links in the navigation bar to log in and

sign up to the application.

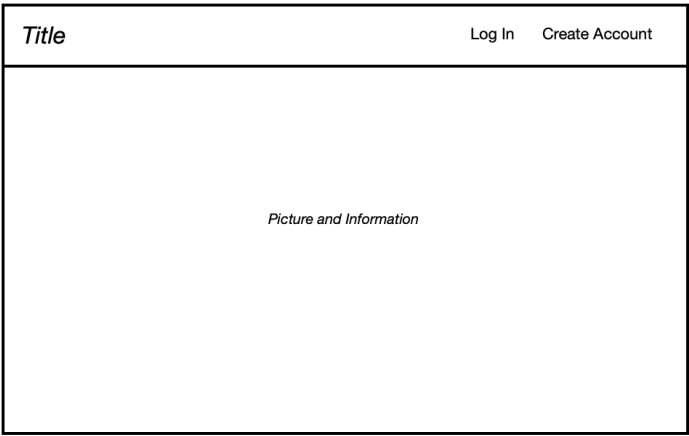


Figure 5.4: Home Page Design

Login

The login page should allow a registered user to log in to the application with valid credentials (username and password) stored in the database. It should prevent unauthorised access to users with invalid credentials. The password field must be hidden when entered (i.e. Password123 should show *****). It should allow redirection back to the home page by clicking the title name in the navigation bar and to the sign up page.

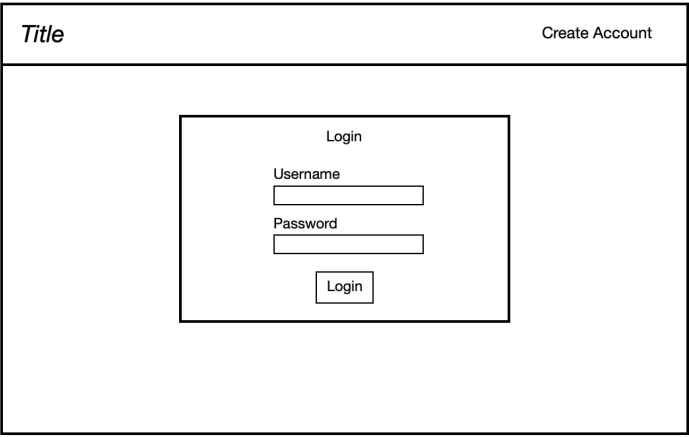
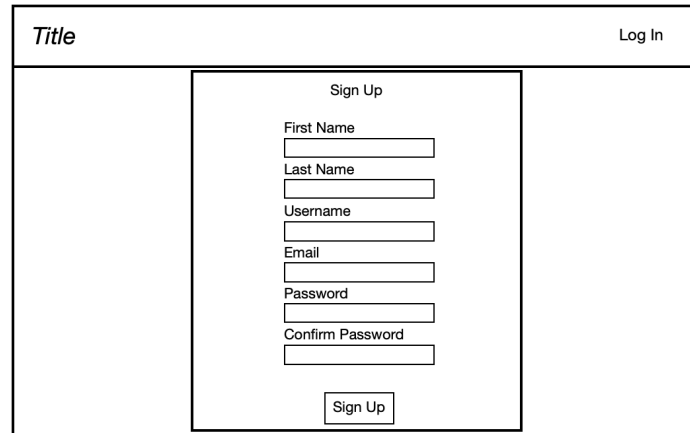


Figure 5.5: Login Page Design

Sign Up

The sign up page should allow a new user to create a new account for the application and log them in on completion. It requires entering the full name of the user, a username, an email, and

a password (with confirmation). Each of these fields should be validated. The first name and last name field should only allow alphabetic characters to be entered, whilst the username and password may contain alphanumeric characters and special characters. The email field must be in the form example@org.com and other acceptable email formats. It should allow redirection back to the home page and to the log in page.



The diagram illustrates the layout of a sign-up page. It features a main container with a header bar at the top. The header bar contains the word "Title" on the left and "Log In" on the right. Centered within the main container is a "Sign Up" form. This form includes input fields for "First Name", "Last Name", "Username", "Email", "Password", and "Confirm Password", each followed by a text input box. A "Sign Up" button is positioned at the bottom of the form.

Figure 5.6: Sign Up Page Design

This covers functional requirements **FR1** and **FR2**.

5.4.2 Dashboard

The dashboard page is what the user sees when they log in. It should contain a navigation bar that provides links to the other pages in the application: the documents page, the loans page, the view properties page, the finance calculator page, the floor plans page, and the information page. It should also provide functionality to edit the user's profile (where they can update their information such as their name, email and password) and to log the user out. This should be contained under the user icon and should be visible upon hovering over it. The aim of this page is to display an overview of the user's portfolios and properties. It should provide statistics such as the total number of properties and rooms, the total value of the properties, and the occupancy rate. It should also provide an informative chart showing an analysis of the cash flow over time, i.e. total income and total expenses. There should also be a reminders / prospective work section where a user can quickly see any reminders or work that needs to be done. They should be able to edit, add, tick off or remove any reminders.

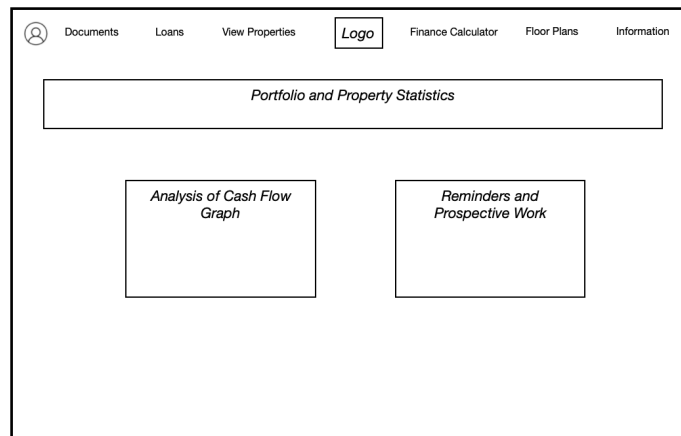


Figure 5.7: Dashboard Page Design

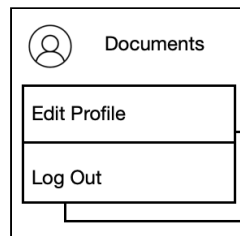


Figure 5.8: User Dropdown Design

This covers functional requirements **FR3**, **FR4** and **FR18**.

5.4.3 View Properties

On entering this page, the user should be able to select from a list of their portfolios. They should also be able to search for a particular portfolio or add a new one. When hovering over a particular portfolio, a user must also have the option to edit or delete the portfolio.

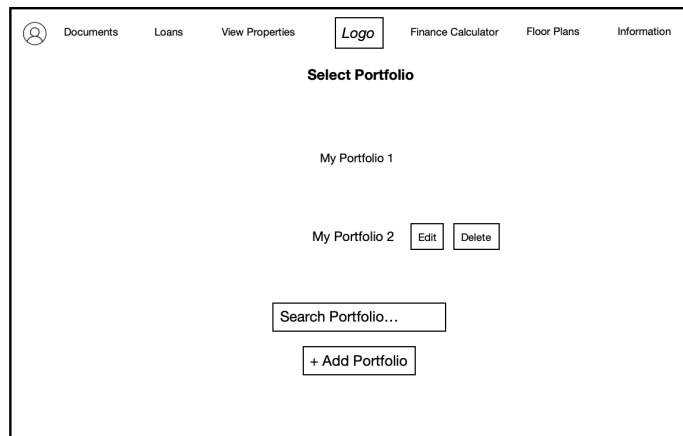


Figure 5.9: View Properties Page Design - Selecting a Portfolio

When creating, editing or deleting a portfolio, popup windows must appear requesting the relevant information and confirming the action.

Figure 5.10: Creating a Portfolio

Figure 5.11: Editing a Portfolio

Figure 5.12: Deleting a Portfolio

This covers functional requirements **FR5**, **FR6** and **FR7**.

When a portfolio has been selected, the user must be able to view the properties associated with that portfolio and select a property to view. They should also be able to search for a

particular property or add a new one. When hovering over a particular property, a user must also have the option to edit or delete the property.

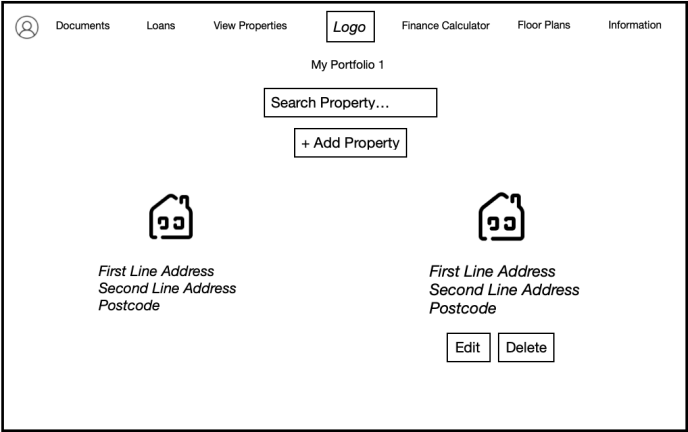


Figure 5.13: View Properties Page Design - Selecting a Property

When a property has been selected, the user must be able to see relevant statistics for that property, such as the total value, the number of rooms, total income and expenses, occupancy, and the relevant licenses. A chart showing income per room and an analysis of cash flow must also be displayed. Additionally, the user must be able to view the floor plan associated to that property. The user must have options to add and edit rooms.

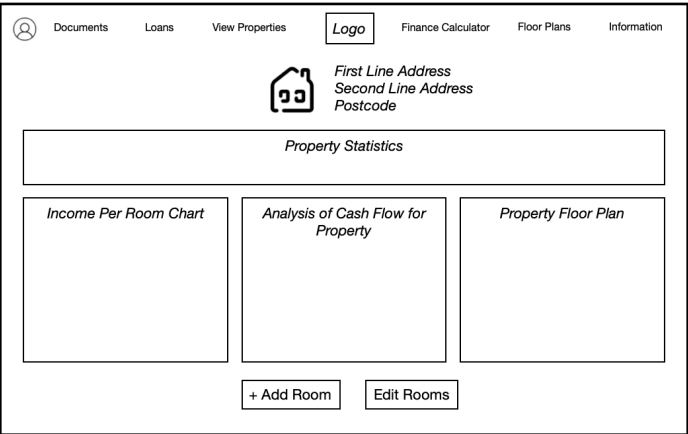
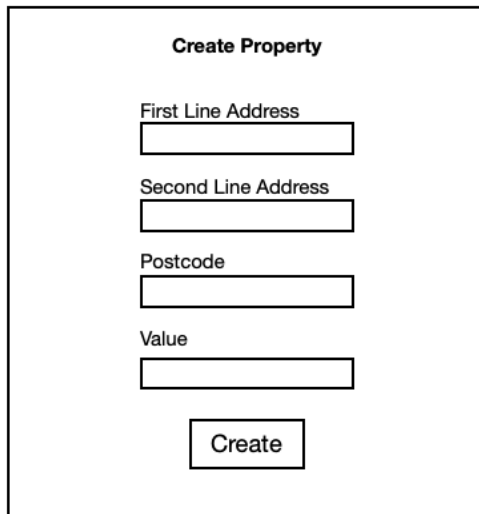


Figure 5.14: View Properties Page Design - Viewing a Property

When creating, editing or deleting a property, popup windows must appear requesting the relevant information and confirming the action.



Create Property

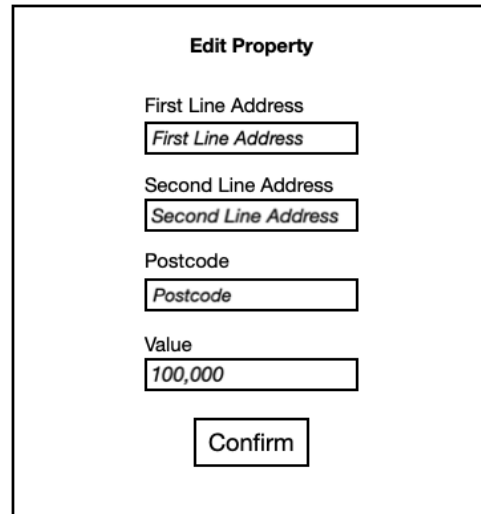
First Line Address

Second Line Address

Postcode

Value

Figure 5.15: Creating a Property



Edit Property

First Line Address

Second Line Address

Postcode

Value

Figure 5.16: Editing a Property



Delete Property

Figure 5.17: Deleting a Property

This covers functional requirements **FR8**, **FR9**, **FR10**, **FR11**, **FR12** and **FR15**.

When creating, editing or deleting a room, popup windows must appear requesting the relevant information and confirming the action.

Create Room

Room Name

Rent

Occupied

Create

Edit Rooms

Room 1	Rent	Occupied	Delete
Room 2	Rent	Vacant	Delete
Room 3	Rent	Occupied	Delete
Room 4	Rent	Vacant	Delete

Confirm

Figure 5.18: Creating a Room

Figure 5.19: Editing a Room

Delete Room

Delete

Figure 5.20: Deleting a Room

This covers functional requirements **FR13**, **FR14**, **FR16** and **FR17**.

5.4.4 Floor Plans

This page allows users to draw, edit, optimise floor plans and save them to a particular property.

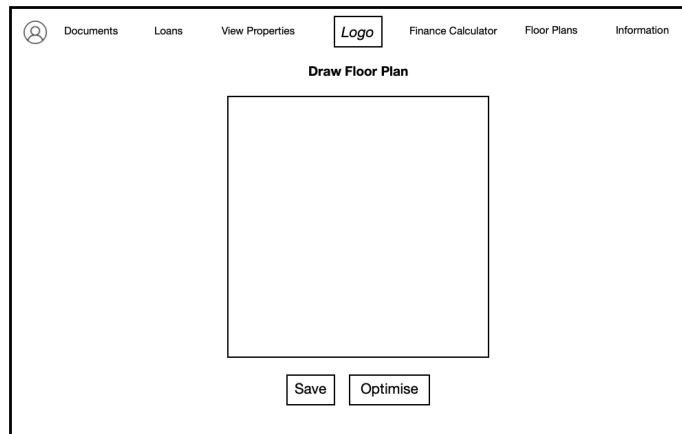


Figure 5.21: Floor Plans Page Design

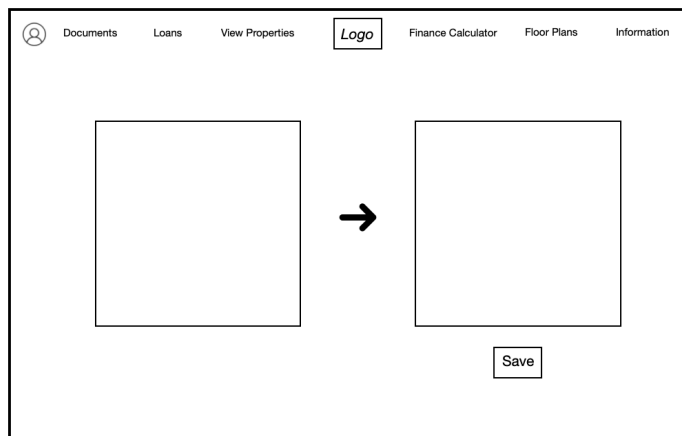


Figure 5.22: Floor Plans Page Design - Optimisation View

Save Floor Plan	
My Portfolio 1	Property 1
My Portfolio 2	Property 2
	Property 3
Save	

Figure 5.23: Floor Plans Page Design - Saving a Floor Plan

This covers functional requirements **FR21**, **FR22**, and **FR25**.

5.4.5 Documents and Information

The documents page should contain all the relevant document regarding a user in respect to their properties. A user should be able to upload, read and download documents. This should be sorted per portfolio / property / room. The information page must include useful information relevant to aiding a user with property management. This can include laws, rules and regulations, information about licenses, current news and trends in the property market. This covers functional requirements **FR10**, **FR14**, and **FR20**.

5.5 Security

Here we consider the various security precautions necessary for the application.

5.5.1 Man-in-the-Middle (MitM) Attacks

We can protect against Man-in-the-Middle (MitM) Attacks by serving exclusively on HTTPS. This will mean that the content will not appear as plain text for a Man-in-the-Middle (MitM) to read and understand. This can be done in Django by configuring a "SECURE_SSL_REDIRECT" so that any HTTP request gets redirected to HTTPS.

However, this is an incomplete solution. A MitM could intercepts the "redirect to HTTPS" response and change it. Additionally, a MitM could intercept the initial HTTP request and create their own HTTPS connection to the server. This would allow the MitM to read all of the unencrypted information whilst both the user and server believe they are communicating securely.

To overcome this issue, the "SECURE_HSTS_SECONDS" setting can be configured in Django. By using HTTP Strict Transport Security (HSTS), we can ensure browsers only use HTTPS. This Django setting defines how long browsers should remember to use HTTPS only, which can be set to long period, for example, 31536000 seconds - 1 year, to ensure protection. Even if the website has misconfigured HTTPS certificates, it will not allow rollback to HTTP, further creating a protective barrier against MitM attacks [36].

5.5.2 Denial-of-Service (DOS) Attacks

We can protect against Denial-of-Service (DOS) attacks using rate limiting. Rate limiting works by limiting the number of requests a user or client can make within a certain timeframe. This prevents the system from crashing from too many requests, since we can set a maximum

number of requests in which the system would not be overloaded. To do this, we can import the `AnonRateThrottle` from the Django REST Framework [37].

5.5.3 SQL Injection

We can protect against SQL injection by using Django's ORM, which prevents having to write raw SQL queries by allowing us to use built-in methods and therefore mitigates any opportunity for injection [38]. Additionally, we can enforce strict validation on inputs that require interaction with the database to further reduce this risk.

5.5.4 Cross-Site Scripting (XSS) Attacks

We can protect against XSS attacks using Django's built-in `"SecurityMiddleware"` setting. Middleware is software that acts as a bridge between different systems, providing common services to facilitate communication and integration. This setting performs a suite of security checks and enhancements to protect against many attacks such as XSS. `"SecurityMiddleware"` sets the `"X-Content-Type-Options"` header to `"nosniff"` which prevents hackers from tricking the website into executing a malicious javascript file that they have uploaded. `"SecurityMiddleware"` also sets the `"X-XSS-Protection"` header to `"1; mode=block"` when `"SECURE_BROWSER_XSS_FILTER"` is `True` to enable the browser's built-in XSS protection [39].

5.5.5 Cross-Site Request Forgery (CSRF) Attacks

We can protect against CSRF attacks using Django's built-in `"CsrfViewMiddleware"` setting. This setting sends a random secret value with a response when the user sends a request, so as to verify the behaviour and thus prevent CSRF attacks in the website [40].

5.5.6 Cross-Origin Resource Sharing (CORS)

CORS is built into the `django-cors-headers` package in the Django REST Framework. This requires downloading and installing the package, including it within the Django's installed apps in the settings and adding the `"corsheaders.middleware.CorsMiddleware"` middleware class. This package and class then allows origins to be specified for which requests are allowed [41]. Here, we must allow our backend server to be a permitted origin to allow communication between the frontend and backend. No other origins should be included to maximise security by preventing malicious sites accessing the system's data.

5.5.7 JSON Web Token (JWT)

Simple-JWT is a library that provides straightforward JWT authentication for Django applications. This requires downloading and installing the library, including it within the Django's installed apps in the settings and configuring the JWT settings. We specify the lifetime of the access token and the refresh token [42]. To make use of the library, we must create views that extend the API class and define authentication classes and permission classes for the relevant views.

5.6 Using Reinforcement Learning

Within real estate management, there are many aspects which could benefit from learning algorithms. One use of learning algorithms in a real estate environment can be to predict the trajectory of a property's value based on various parameters such as number of room, total area, location, etc. This could give real estate owners insights into whether a property is a good investment from a long term appreciate perspective. If a property's predicted value is low or negative, it can recommend owners to divert funds to another property or another source of investment. If the predicted value is high, it can inform an investor of potential high-reward investments. It can also give insights on how much an investor can borrow against the property by revaluing and remortgaging the property.

Another use case for learning algorithms in a real estate perspective is calculating the optimal rent prices for a property or for individual rooms based on the amenities it provides, the location and the size. This would allow a real estate manager to strategically place rent prices to optimise total income. It could also provide insights on better performing rooms or properties and analyse how to increase occupancy and consequently turnover.

Whilst many aspects could provide substantial benefit to a real estate owner, manager or investor, the focus of this project is on architectural optimisation in terms of floor plan designs. By optimising space and minimising costs in this way, a learning algorithm can provide considerable, valuable information on how to increase the internal value of a property. It would boost both the entire value of the property and also rent prices, producing higher returns on investment. By adhering to legal compliance set out by housing regulations, it additionally aims to improve conditions for tenants.

Reinforcement Learning (RL) encompasses a wide range of algorithms that can be used towards

this optimisation problem. This includes algorithms such as Proximal Policy Optimisation (PPO), Q-Learning, Monte-Carlo Tree Search (MCTS) and Multi-Agent Deep Deterministic Policy Gradient (MADDPG).

Whilst Q-Learning is simple to implement for grid-based layouts and can handle large state spaces with Deep Q-Networks, it risks overestimating Q-values and making major changes.

MCTS is good at combining multiple parameters together, which is essential for this project, however, it is computationally expensive for large spaces. MADDPG makes use of multiple agents for training, which could be useful with multiple stakeholders and deciding trade-offs between the different parameters, but this becomes extremely complex to train and stabilise.

PPO provides the best balance between stability and efficiency in comparison to other algorithms. It works well with both discrete and continuous actions, which would facilitate exact wall positioning and proficiently incorporates parameters as well as rewards. Whilst this requires careful parameter tuning, it presents the best option for this optimisation problem due to its restriction in making major changes, which will ultimately allow for precise, optimal floor plans.

The PPO library from Stable-Baselines3 requires a definition of the policy network architecture. Stable-Baselines3 includes several built-in network architectures such as the MLP policy - a fully connected neural network, Convolutional Neural Networks (CNN) policy- specifically designed for image-based inputs, multi input policy - designed to handle heterogeneous observation spaces with different data types and custom policy networks. Whilst custom policy networks may provide a higher level of flexibility in creating the agent for specialised needs, it becomes extremely complex to correctly configure, requires precise hyperparameter tuning and has great computational cost. This also applies to Multi Input Policy which can allow for more complex environment representation and process each input type with the appropriate network architecture. For floor plan design, whilst this could be beneficial when considering a large amount of parameters, the trade off is a very high level of complexity. Whilst Convolutional Neural Networks (CNN) policy is capable of capturing spatial relationships in input data, it is not ideal for non-image input. Whilst floor plans can be represented as images, this will make them much harder to manipulate. The policy network chosen is MLP policy. Whilst this policy is less effective for high-dimensional inputs like images, it is a simple, well-tested and computationally efficient network that can handle both continuous and discrete action spaces.

Chapter 6

Implementation & Testing

This section explores the process of implementing the design into a functional web application.

6.1 Methodology

The time frame provided for this project necessitated an agile approach to software development. Agile software development essentially requires breaking down the project into small units, and providing frequent updates whilst accommodating for changing requirements. It focuses on continuous improvement and delivery, adjusting the work in sections to approach the final desired result, finding the simplest working solutions to achieve this throughout [43]. This enabled an iterative development towards the project, always ensuring a working solution was available, with a strong, initial focus on the primary requirements. In this way, extending the project with changing requirements becomes more manageable and enables it to be built up in a short amount of time.

The two fundamental parts of the project - real estate management and architectural optimisation - works well with this modular approach. Prior to the implementation of this project, the system requirements were defined as outlined in the Chapter 3. Following this, these requirements were met within separate development sprints, a short time-period in which you aim to complete a set amount of work. After each sprint, the results are validated against the initial requirements to monitor effectiveness and are subject to review. If any changes need to be made, this is requested in the review. This process is repeated until the project has been completed. By proceeding in this way, the real estate management portion of the project was created as a foundation for the architectural optimisation. Whilst the architectural

optimisation portion of the project provides distinct features, this approach provided seamless integration with the real estate management portion.

6.2 Code Organisation

The source folder is separated into two folders: the frontend and the backend.

6.2.1 Frontend

The frontend folder contains the code for the frontend portion of the project. This includes the configuration of Vite, JavaScript and JavaScript XML (JSX) files which allow us to create React elements, connect Vite to the Django backend, map pages to URLs and provide interactivity to the elements on a web page. It also includes Cascading Style Sheets (CSS) which provides styling to the web pages and all images used in the application.

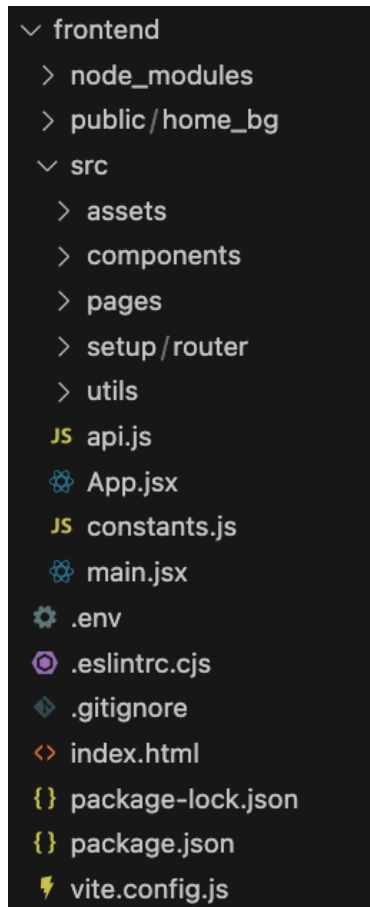


Figure 6.1: Frontend Folder Structure

node_modules folder

This folder contains all the packages imported via the NPM.

public folder

This folder contains all the files that are publicly accessible over the network from the project.

src folder

This folder contains all the frontend code.

assets folder

This folder contains all the images used within the web application.

component folder

This folder contains React components. A React component is a UI element that can be re-used across multiple different pages or the same page.

pages folder

This folder contains the code for each page / view of the web application. It is separated into further subfolders for each page which contains the JSX code for the page, any specific components and specific CSS styling.

setup/router folder

This folder contains a router which maps URLs to specific web pages.

utils folder

This folder contains code including helper functions for the entire application.

api.js

This JavaScript file initiates Axios to provide a connection between the frontend and backend. It also provides an access token for a user when they interact with the backend using the frontend.

App.jsx

This JSX file defines the template, React code for the web application.

constants.js

This JavaScript file defines any constants used in the entire application.

main.jsx

This JSX file is the code that is first run when the frontend is run. It creates the React DOM to allow the other React pages to run.

.env

This file defines the pathway or URL to the backend.

.eslintrc.cjs

This file is a configuration file used to set the rules for ESLint. ESLint is a popular JavaScript code linter that helps developers identify issues within their code, maintaining code quality and consistency.

.gitignore

This file stores the name of files that should not be pushed to the repository when committing changes.

index.html

This file is the HTML file which gets run by the backend. It facilitates the running of React in a format compatible with the backend.

package-lock.json

This file is generated and updated automatically by the NPM when installing or updating packages. It is used to lock the exact versions of dependencies installed in the project, ensuring reproducibility and consistent installations across different environments.

package.json

This file is primarily used for managing and documenting metadata about the project, including its name, version, author, dependencies, scripts, and other configuration details.

vite.config.js

This JavaScript file configures Vite.

6.2.2 Backend

The backend folder contains the code for the backend portion of the project. This includes the configuration of the API between the frontend and backend, defining URLs and views for the backend code, the models and forms.

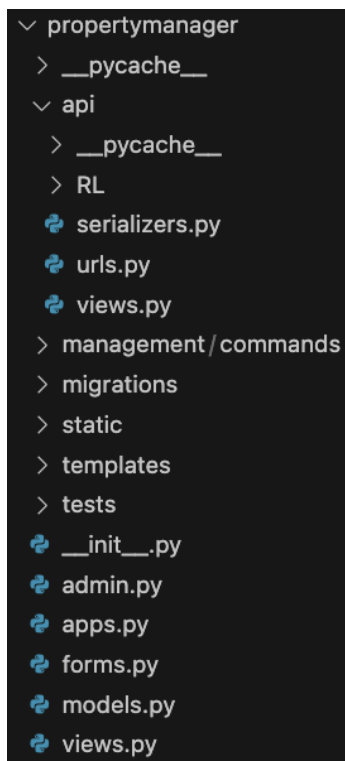


Figure 6.2: Backend Folder Structure

__pycache__ folder

This folder stores compiled code of imported modules to facilitate faster loading.

api folder

This folder contains the code for the API that formats data to be compatible with the frontend.

RL folder

This folder contains the code for training and running the Reinforcement Learning (RL) agent for floor plan optimisation.

serializers.py

This file contains the code for the serialisation of backend data to frontend-compatible data.

urls.py

This file contains the URL mapping for the backend code.

views.py

This file contains the code that defines how the application interacts with users' requests.

management folder

This folder contains custom Django admin commands.

migrations folder

This folder contains a blueprint of all changes in models.

static folder

This folder holds any assets, such as images, CSS and JavaScript files, that are used by the backend.

templates folder

This folder holds HTML files that define web pages used by the backend.

tests folder

This folder contains unit tests for application components.

__init__.py

This file transforms a directory into a Python package.

admin.py

This file configures how the application's models are presented in Django's admin interface.

apps.py

This file manages application-specific configurations.

forms.py

This file contains form classes for data input.

models.py

This file defines data structures using Django's ORM, that will be used in the database.

6.2.3 Source

The source folder also contains other important files such as the Django settings, the `manage.py` file and the database.

manage.py

This file stores various Django management commands, such as initiating the development server, creating an application and running migrations.

settings.py

This file contains the settings that configure the Django project.

wsgi.py

WSGI is short for Web Server Gateway Interface. This file serves as the entry point for the application when deployed on a production server. It bridges the application to the web server, enabling it to handle incoming requests.

asgi.py

ASGi is short for or Asynchronous Server Gateway Interface. This file serves as the entry point for asynchronous web servers and facilitates the handling of asynchronous HTTP requests.

db.sqlite3

This is the database that stores data in the form of the models outlined in the Django models file.

requirements.txt

This file contains a list of packages or libraries (dependencies) required for a project. This allows for straightforward installation of all dependencies. It provides a consistent environment and facilitates better collaboration.

6.3 System Functionality

The implementation of the system functionality for this project began with configuring the backend as a foundation for the frontend and to ensure security.

6.3.1 Initial Backend Configuration

The initial backend configuration involved creating a template for the application. This consisted of connecting the backend to the frontend and providing security features.

Within the source folder, the `__init__.py`, `settings.py`, `urls.py`, `wsgi.py`, `asgi.py` files and the database are configured automatically by Django using the "django-admin startproject" command in the terminal. This defines the root URL of the project, the backend template, the secret key for the application, the WSGI application, the database, the language and time zone, the static url, the installed apps that the application uses and the middleware within the `settings.py` file. The `wsgi.py` and `asgi.py` files are configured to deploy a server and allow requests to the server. The `urls.py` provides a blank template for adding URLs to the application. Finally it creates a `db.sqlite3` file which is the database.

The `admin.py`, `apps.py`, `models.py`, `tests.py`, `views.py` files and the migrations folder are configured automatically by Django using the "python3 manage.py startapp" command in the terminal. This gives us a blank template for these files.

6.3.2 NPM dotenv

npm dotenv is a module that loads environment variables from a .env file [44]. Storing sensitive information such as API keys and database passwords in a .env file is a secure practice. npm dotenv ensures these credentials are loaded into the application without being exposed in the codebase or version control. This practice is crucial for maintaining the security of the application [45].

To use the npm dotenv module, the `load_dotenv` function is imported from the module and run within our `settings.py` file. This allows us to read credentials from the .env file, loading all variables to the environment and storing them into Python variables [46].

6.3.3 Security

As mentioned in the design of the project, there are various security precautions that must be ensured.

Cross-Origin Resource Sharing (CORS) is enabled for added protection. This is achieved through the insertion of the `'corsheaders.middleware.CorsMiddleware'` middleware to the `MIDDLEWARE` section in the `settings.py` file. This requires defining the trusted and allowed origins which is set to our frontend URL and by allowing credentials to CORS. To allow this configuration to work with the application, we add `corsheaders` to the `INSTALLED_APPS` section in the `settings.py` file. This enables the backend to communicate securely with the frontend, and prevents malicious sites from read the application's data.

To protect against MitM attacks, the `'django.middleware.security.SecurityMiddleware'` is added and the following settings are set: `SECURE_SSL_REDIRECT = True` and `SECURE_HSTS_SECONDS = 31536000`. This allows the web application to run solely on HTTPS and prevents hackers from intercepting requests.

To protect against DOS attacks, default throttle classes and default throttle rates are added to the Rest Framework setting. These allow for specifying rate limits for anonymous users, i.e. unregistered users and registered users. The rate for anonymous user is set at 100 per day and the rate for registered users is set to 1000 a day. This helps in controlling the number of requests sent by a user within a day. An unregistered user is more likely to be initiating a DOS attack and so have a much stricter limit. The relevant throttle classes must be specified with each view, seen later.

To protect against XSS attacks, `SECURE_BROWSER_XSS_FILTER = True` is specified in the Django settings to enable the browser's built-in protection, preventing hackers from

uploading malicious scripts to the application.

To protect against CSRF attacks, the `'django.middleware.csrf.CsrfViewMiddleware'` middleware is added to the settings file. This automatically verifies the user making a request.

6.3.4 Initial Frontend Configuration

To set up the frontend the `"npm create vite@latest --template react"` command is used in the terminal to configure Vite with React installed. We also install Axios, react-router-dom and jwt-decode using the `"npm install axios react-router-dom jwt-decode"` command. React-router-dom facilitates travelling to different URLs in React and jwt-decode enables the decoding of JWT tokens. This produces the `node_modules` folder, the `public` folder, the `src` folder as well as the `.eslintrc.cjs` file, the `.gitignore` file, the `index.html` file, the `package-lock.json` and `package.json` files and the `vite.config.js` file within the frontend folder.

Following the configuration of Vite, a `constants.js` file is created which contains constants for the access and refresh tokens. This defines a key for accessing the tokens that will be stored in local storage. This facilitates easier retrieval of the tokens.

Next an `api.js` file is created. This file includes code for an interceptor, which intercepts any sent request and automatically adds the correct header to prevent repetitive code and easier maintenance. This uses Axios to check if a user has an access token and automatically send it to requests. This process is completed by importing variables within the `.env` file in which we define the URL of the backend server and check if the user has an access token in local storage. If there is an access token in local storage, this is added as an authorisation header to a request.

A "protected route" is then created, which is essentially a URL which requires authentication to access. This is completed by creating a `ProtectedRoute.jsx` file which checks if a user is authenticated to access a path, i.e. is logged in, which allows them to enter a protected route if logged in, or be navigated to the log in if a user attempts to access a page which requires authentication without being logged in. This function works by checking if a user has an access token in local storage. If they do not have an access token in local storage, the system will check if they have a refresh token. If a user has neither token, they are navigated to the log in page. If a user has a refresh token and an expired access token, the access token is refreshed by decoding the access token and updating its expiration time. If a user has a refresh token but not an access token, a request is sent to the backend to get a new access token. If a user has both tokens, they are navigated to the desired page.

6.3.5 Login and Sign Up

Login:

Creating the User Model

Within the `models.py` file in the backend, we create a model of a user. This includes the username, first name, last name, email and password of the user. Additional validation is added here as specified in the design, specifically, requiring a username to have a minimum of three alphanumeric characters, requiring the first name and last name to exclusively comprise letters and requiring the email to be in the format `x@x.x`.

Backend Authentication Configuration

Within the `settings.py` file, the Django REST Framework is configured, specifically the default authentication classes and the default permission classes. The default authentication classes is a list of authentication classes that determines the default set of authenticators used when accessing the `request.user` or `request.auth` properties. The `request.user` field identifies which user is sending a request and the `request.auth` field identifies additional authentication information, such as a token. The default permission classes is a list of permission classes that determines the default set of permissions checked at the start of a view [47].

For token-based authentication of a user, the Simple JWT plugin is used. Simple JWT provides a JSON Web Token (JWT) authentication backend for the Django REST Framework. In `settings.py`, we define the lifetime of the access and refresh tokens using Simple JWT. The lifetime of the access token is at 30 minutes and the lifetime of the refresh token is set to 1 day. The lifetime of the access token is relatively short since if it is compromised, it expires relatively quickly and so prohibits how long an unauthorised user can use it. When the refresh token expires, the user must log in to the application again to generate new tokens. This provides secure user authentication.

To allow this configuration to work with the application, `rest_framework` and `rest_framework_simplejwt.token_blacklist` is added to the `INSTALLED_APPS` section of the `settings.py` file.

Serialising the User Model

As previously mentioned, it is a requirement to convert Python objects to the JSON format and vice versa to be able to communicate with the web application and receive a response back. To do this, a serializer is created within the `serializers.py` file for the User model. The serializers

library from `rest_framework` facilitates this.

Since a user will log in only with a username and password, but requires more input for signing up, this requires a separate serializer for logging in and signing up. Here we implement the login serializer.

To enable the above, a `LoginSerializer` class is instantiated that inherits from the `ModelSerializer` from `serializers` library, and defines the `User` model and the fields (in this case just the username and password). The password is defined as write only, i.e. should be sent to the server to verify but shouldn't be sent back to the frontend in the response.

Defining the Login URL

In the `urls.py` file, we define a new url for logging in by entering the URL path, the `TokenObtainPairView` view imported from the `simplejwt` library within `rest_framework` and a name for the path. The `TokenObtainPairView` allows the server to get an access token and refresh token as a pair.

We create an additional URL for refreshing the access token by entering a new URL path, the `TokenRefreshView` view imported from the `simplejwt` library and a name for this path. The `TokenObtainPairView` facilitates refreshing an access token.

Finally, we implement an additional URL which include URLs from the REST framework to facilitate authentication.

This allows a user to create an access token and refresh token, to refresh their access tokens automatically and essentially log in using the URLs.

Creating the Login Page

After setting up the backend configuration for the login, the login page is created using React. As per the design, this includes a navigation bar which contains the title and a link to redirect to the sign up page, and a form asking the user for their username and password. This form was developed as a separate component which was imported by the main login page file to improve maintainability. The login form creates an input for the username and password, and assigns the value of the input to a field for each input. The form contains a log in button which sends these input fields to the backend to verify the credentials. If the credentials are valid, the function then sets an access token and refresh token received by the response from the backend in local storage and navigates the user to the dashboard. If unsuccessful, the error is presented to the user.

To be able to navigate to this page, a route needs to be established on the frontend for it. This task is executed by creating a `pages.tsx` file (a TypeScript Execute (TSX) file), which holds the name for the React pages with corresponding links. Thereafter, a `router.jsx` file is created, which uses the `Route` component from the `react-router-dom` package installed during initial configuration providing the ability to define a path for the frontend pages. This is done by defining a path, set in the `pages.tsx` file, and providing an element - the React login page.

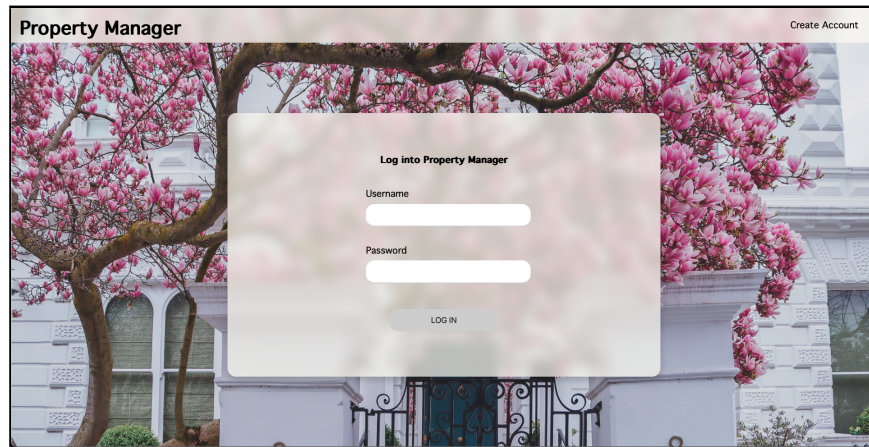


Figure 6.3: Login Page Implementation

Sign Up:

Serialising the User Model

Here, within the `serializers.py` file, we create a `SignUpSerializer` which inherits from the `ModelSerializer` from `serializers` library, and define the `User` model, the fields (in this case all of the user fields: `username`, `password`, `first name`, `last name` and `email`) and we define the `password` as write only. We additionally create a new function that enables the creation of a new user. This function takes in some data which has been validated by the serializer and creates a new `User` object using this data.

Creating the Sign Up View

Following model creation and serialisation, we create a class-based view to create a path for signing up. We do this by inheriting from the `CreateAPIView` from the `generics` library as part of the `rest_framework` to create a `CreateUserView`. Here we define the query set, the serializer class and the permission classes. The query set is a list of all users which the `rest_framework` uses to prevent duplication of an object, in this case, a user. The serializer class refers to

the serializer produced in the serializers.py file - here, the SignUpSerializer, which tells the view what type of data we need to accept to create this object. The permission class refers to who can call this method, which is set to AllowAny which means anyone can call the method. This is necessary to allow any user to be able to create an account, not one that is necessarily authenticated.

Defining the Sign Up URL

In the urls.py file, we define a new url for signing up by entering the URL path, the CreateUserView view created in the views.py file and a name for the path. This allows a user to call the create user method within this class using the URL.

Creating the Sign Up Page

As per the design, the sign up page includes a navigation bar which contains the title and a link to redirect to the login page, and a form asking the user for their username, first name, last name, email, password and a confirmation of their password. The form contains a sign up button which sends these inputs to the backend sign up URL which calls the CreateUserView class and the create_user method which validates the data and creates a new user in the database if valid. If successful, the user is redirected to the login page where they can log in. If unsuccessful, the error is presented to the user.

At this stage, we create a route to the sign up page in the router.jsx file and define a path for the sign up page in the pages.tsx file. This route is wrapped within a function that first clears local storage of any tokens before a user attempts to sign up to prevent any token conflicts when creating a new user.

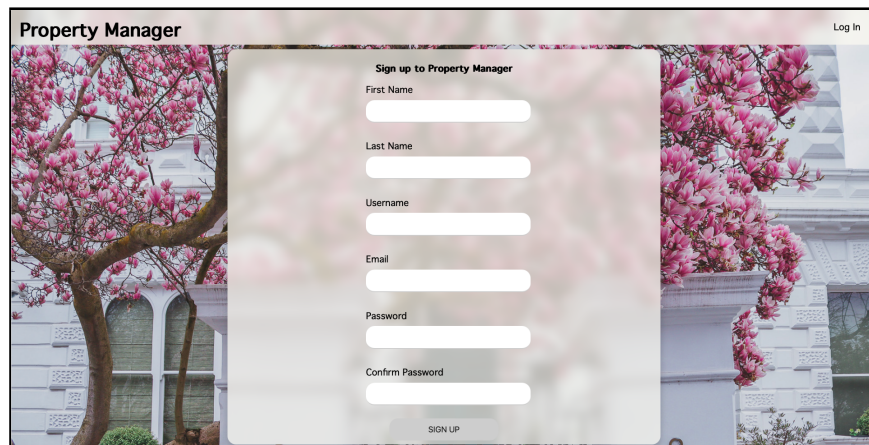


Figure 6.4: Sign Up Page Implementation

Making Migrations:

To allow changes to our models to carry across to the database, we go through a process called migrating. To execute this, we run the "python3 manage.py makemigrations" command in the terminal to make the migrations followed by the "python3 manage.py migrate" command to apply the migrations.

6.3.6 Home, Log Out and Not Found

Creating the Home Page

As per the design, the home page includes a navigation bar which contains the title and links to redirect to the login and sign up page. A path for this page is defined in the pages.tsx and a route is defined in the router.jsx file using this path. This page features an animation on scroll which zooms into to different parts of a property and provides information about the application.



Figure 6.5: Home Page Implementation

Creating the Not Found Page

Whilst not designed, a not found page would be a beneficial part in making the system look consistent and professional. A user would be directed to this page if they attempt to access a URL that has not been defined. The implementation of this page is a simple page consistent with the other pages of the application stating that the page was not found.

Enabling Logging Out

To enable logging out, we include a log out function in the router.jsx file which clears the local storage of any access and refresh tokens and redirects them to the login page. We additionally create a view in the backend views.py file that retrieves any access or refresh tokens the user currently has active and "blacklist" them, essentially rendering them unusable. This prevents them from accessing any authenticated pages, effectively logging them out from the system.

6.3.7 View Properties

This page allows the user to view, edit, delete their portfolios, properties, rooms and provide information for each.

6.3.8 Creating the Models

Creating models for portfolios, properties and rooms will allow them to be stored in the database and be accessed by the relevant user at any time. This objective is attained in the models.py file. For the portfolios, we specify a foreign key on the User model for the user and specify a title. A foreign key provides the primary key of another database table to be a field in the current table. The primary key is the unique identifier for a database record - SQLite assigns a unique identifier for each field as the primary key. For the Property model, we specify a user and portfolio as foreign keys, a first line of address, a second line of address, a postcode, the value, monthly expenses, renting type and licenses. For the Room model, we specify a user and property as foreign keys, and the name, rent and whether it is occupied. The foreign keys have been set up to delete on cascade. This means if a user has been deleted, all corresponding portfolios, properties and rooms will be deleted. If a portfolio has been deleted, all corresponding properties and rooms will be deleted. If a property has been deleted, all corresponding rooms will be deleted.

6.3.9 Serialising the Models

To use them in the frontend, we create serializers for each model specifying all their relevant fields.

6.3.10 Creating the Views

As per the design, we require creation, modification and deletion function for the Portfolio, Property and Room model.

To enable the creation methods, we inherit the `ListCreateAPIView` class from the generics library in `rest_framework` which handles the creation in the database, we define the serializer class and the permission classes. The permission class here is `IsAuthenticated` which allows only authenticated users to interact with these models. This class also defines a method to retrieve the query set and perform a creation of the object using valid data.

To enable the modification methods, we inherit the `UpdateAPIView` class from the generics library in `rest_framework` which handles the modification in the database, we define the serializer class and the permission classes. The permission class here is `IsAuthenticated`. This class also defines a method to retrieve the query set.

To enable the deletion methods, we inherit the `DestroyAPIView` class from the generics library in `rest_framework` which handles the deletion in the database, we define the serializer class and the permission classes. The permission class here is `IsAuthenticated`. This class also defines a method to retrieve the query set.

6.3.11 Creating the URLs

We define URLs for all the views. For the modification and deletion views, the system needs to know which portfolio, property or room to perform the action on and so in the URL, we must specify this. To do this, we pass in the portfolio ID to the portfolio modification and deletion views, the property ID to the property modification and deletion views and the room ID to the room modification and deletion views.

6.3.12 Creating the Page

On entering this page, the user is prompted to select a portfolio. This is retrieved from the database by sending a request to the backend. The user additionally has the option to search through their portfolios or add a new one. This page contains a navigation bar to all other pages in the application which is created as a component to enable reuse throughout the application.

The navigation bar also features the option of dark mode, extending the original design, which users can toggle to change the system theme to a darker colour. This can provide reduced eye strain in low light-conditions, improved accessibility to make text easier to read for users with visual impairments, reduced blue light exposure and a more visually appealing experience.

The buttons that appear next to a portfolio name when hovering over it have been modified from the design to provide a more minimalist, aesthetic and professional look. The edit popup has also been discarded in favour over a sleeker, seamless integration of editing the portfolios by simply encapsulating an existing portfolio in a text box, with the option to confirm or cancel.

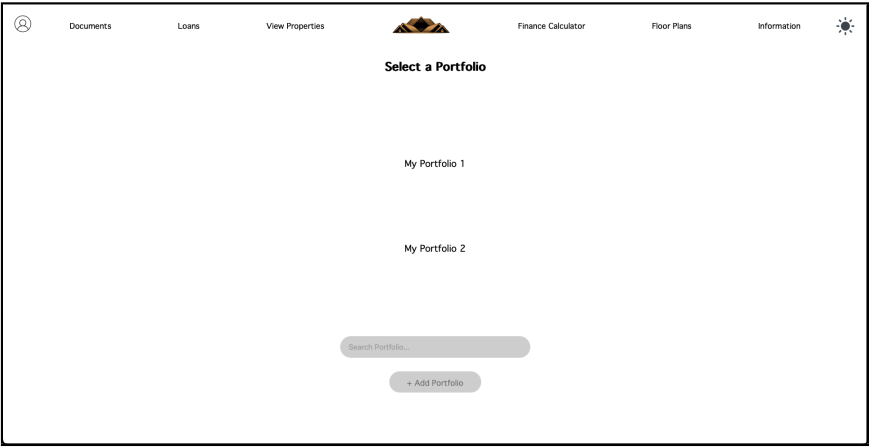


Figure 6.6: View Properties Page Implementation

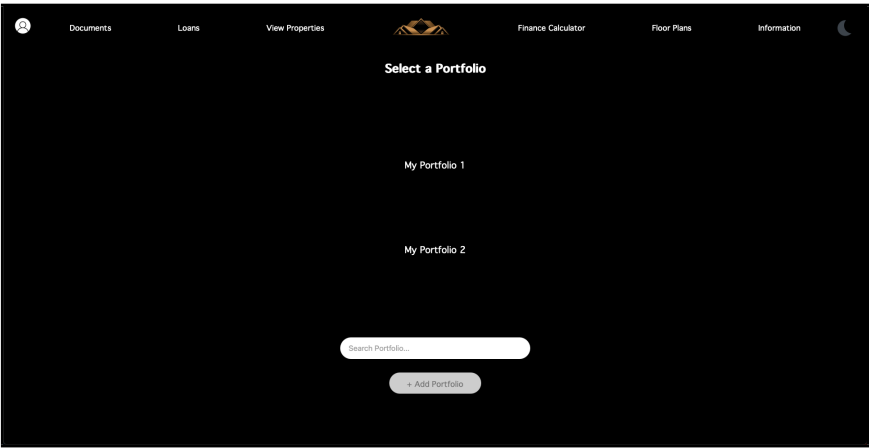


Figure 6.7: View Properties Page - Dark Mode



Figure 6.8: View Properties Page - Hovering Over a Portfolio



Figure 6.9: View Properties Page - Editing a Portfolio

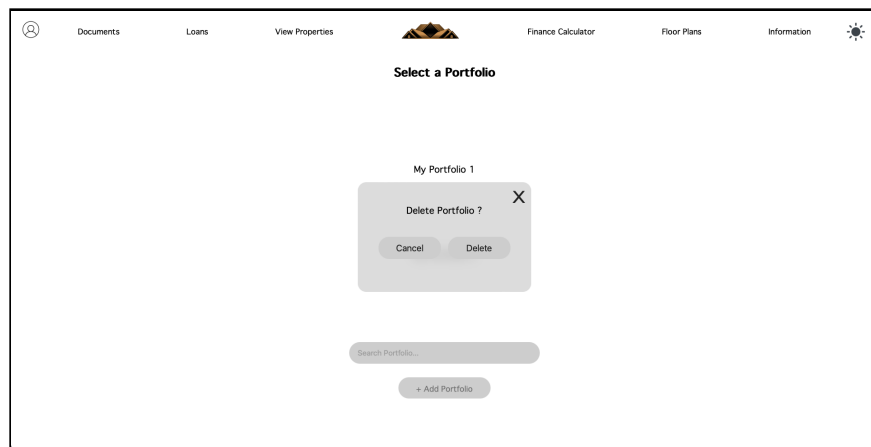


Figure 6.10: View Properties Page - Deleting a Portfolio



Figure 6.11: View Properties Page - Creating a Portfolio

To add or edit a portfolio, the React page sends a request with the new portfolio name and the user making the request to the backend that calls the `CreatePortfolioView` or `UpdatePortfolioView` class respectively by running the corresponding URL. To delete a portfolio, the page sends a request with the portfolio ID to the `DeletePortfolioView` class.

When a user selects a portfolio, they have the option to view the properties associated with that portfolio, to select a property, to search through the properties, to add a new property to the portfolio, or to redirect back to all portfolios by selecting the portfolio name. The properties have a similar functionality in terms of editing and deleting on hover. To add or edit a property, the React page sends a request with the new property information and the user making the request to the backend that calls the `CreatePropertyView` or `UpdatePropertyView` class respectively by running the corresponding URL. To delete a property, the page sends a request with the property ID to the `DeletePropertyView` class.

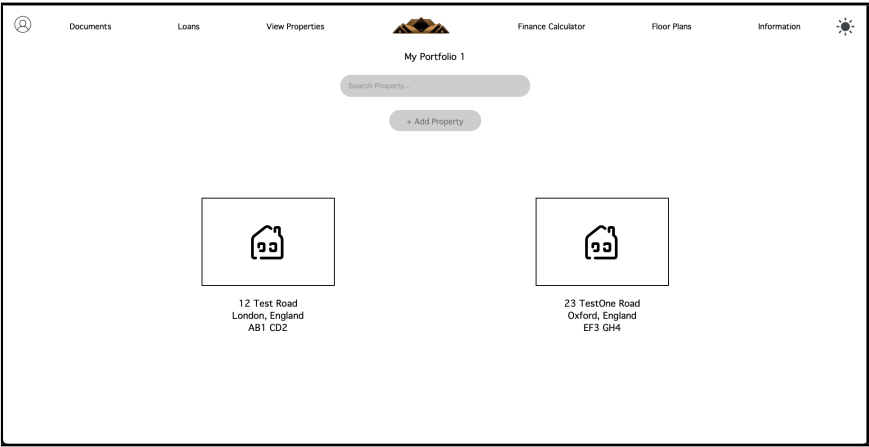


Figure 6.12: View Properties Page - View Properties in Portfolio

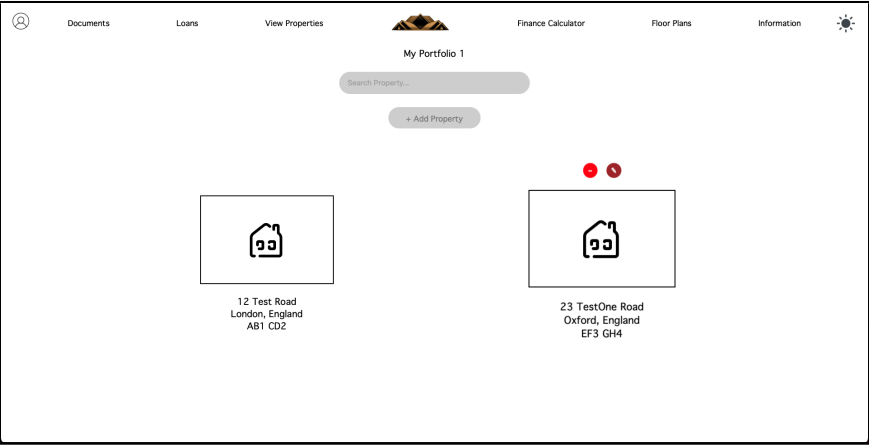


Figure 6.13: View Properties Page - Hover Over a Property

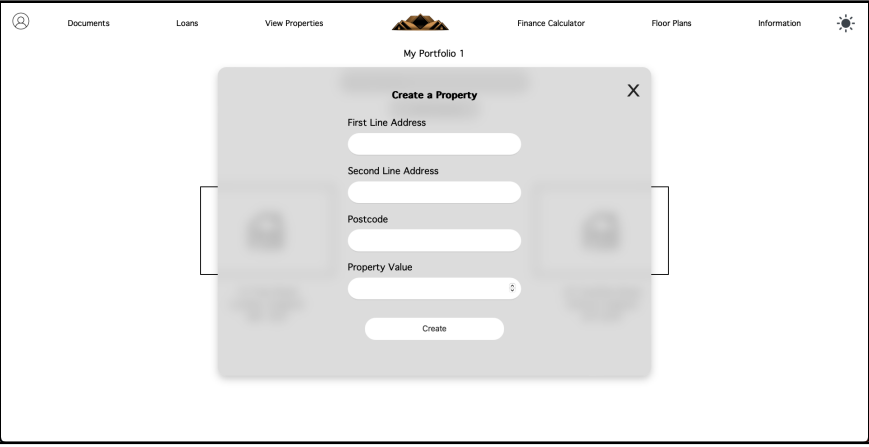


Figure 6.14: View Properties Page - Creating a Property

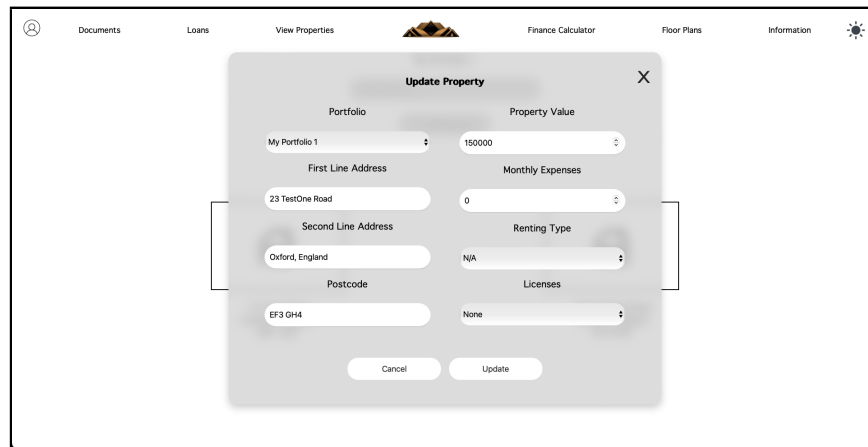


Figure 6.15: View Properties Page - Editing a Property



Figure 6.16: View Properties Page - Deleting a Property

The implementation of editing a property has been extended from the design to allow functionality to edit the monthly expenses, the renting type and the licenses of a property which are set to default values when creating a property. This approach prevents overwhelming the user when creating a new property but ensures they have the option to modify these fields.

Finally, this page allows a user to select a property to view the relevant information concerning only that property. This provides statistics such as the property's total value, the number of rooms, the number of occupied rooms, the total income, the monthly expenses, the renting type and the licenses it holds. It displays the income per room as a pie chart, a graph indicating the monthly income - expense cash flow for the property and the layout of the floor plan if one has been created for the property. It also has options to add or edit rooms.



Figure 6.17: View Properties Page - Viewing a Property

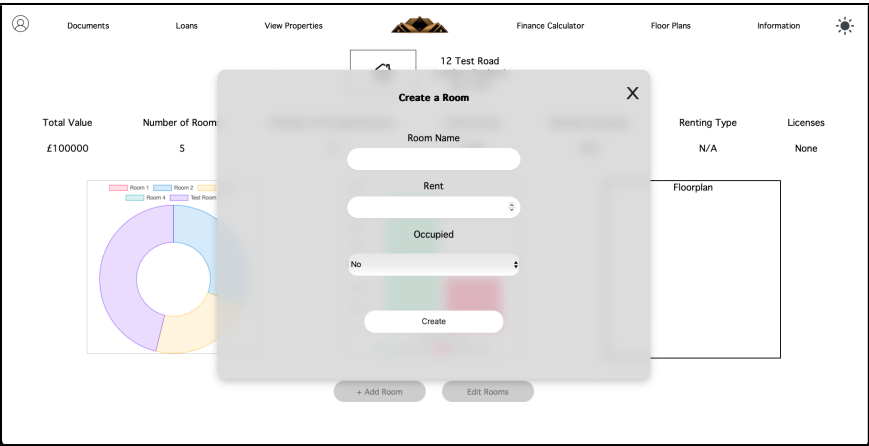


Figure 6.18: View Properties Page - Adding a Room

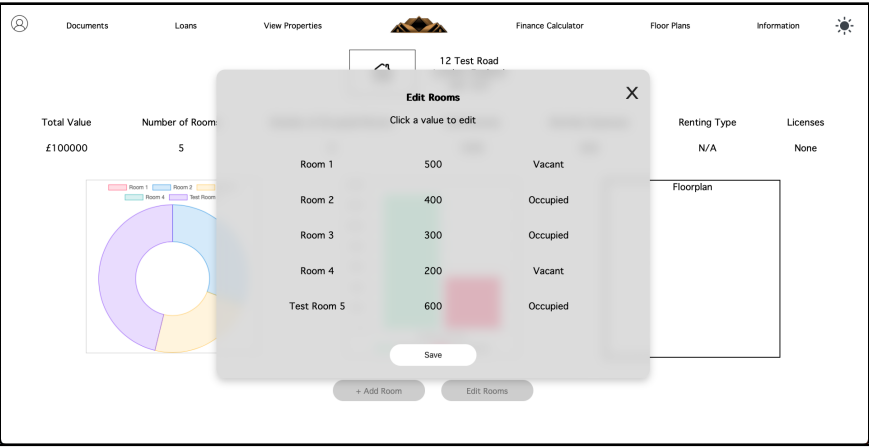


Figure 6.19: View Properties Page - Editing Rooms

6.3.13 Dashboard

The function of this page is to show statistics which represent an overview of the user's properties, as well as providing functionality to keep note of prospective work and other reminders. The statistics shown are the total value of all properties, the total number of properties, the total number of rooms, the occupancy rate and the vacancy rate.

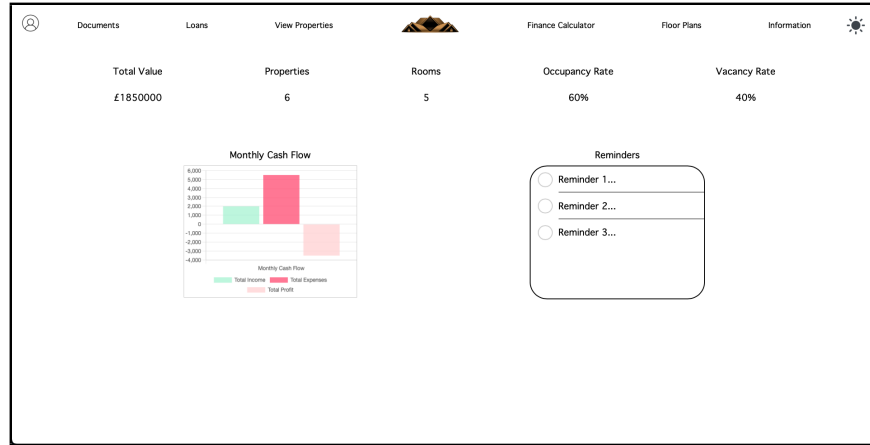


Figure 6.20: Dashboard Page Implementation

Testing

The testing of forms and models described above were done in Python using the Django test library to ensure full functionality and validation, defining test cases for each form, model and input.

6.3.14 Floor Plans

The purpose of this page is to create, edit and optimise floor plans.

Backend Logic:

The floor plan optimisation process trains a Proximal Policy Optimisation (PPO) Reinforcement Learning (RL) agent from a training set of pairs of floor plan designs. These floor plan design pairs represent an original floor plan and the same floor plan after a transformation has been made that optimises space in an efficient way. The agent works by iteratively moving walls within the original floor plan and comparing similarity to produce a reward so it can learn how best to optimise space. This model is then saved so can be used on new floor plan designs to optimise space.

Floor Plan Class

To implement this, we first create a class for a floor plan, which encapsulates all the information needed within a floor plan and the methods for it. This class stores the floor plan boundary coordinates, the entrance coordinates, the rooms as a list and the total area. Here, we also initialise the minimum bedroom size (6.51) and minimum kitchen size (7) as outlined in the background. Whilst these sizes present an over-simplification of the regulations set out by the relevant compliance and may not be directly applicable for all floor plan designs, it provides a strong foundation for training the agent. We also include fields to define the size of rendered floor plans using the matplotlib library.

Room Class

The Room class contains the room type, the room coordinates, the entrance coordinates for the room and any linked rooms.

Room Types

The RoomTypes enum defines the different types of room available and the corresponding colour for them. The room types include a corridor, living room, kitchen, bathroom, bedroom, ensuite, dining room, utility room, stairs and study.

Floor Plan Class Methods

The FloorPlan class contains methods to render the floor plan, to calculate the shoelace area, to move a wall, to find adjoining walls, to add a room, and to remove a room.

The render method uses the matplotlib library to plot the coordinates of a floor plan, including its boundary, entrances and rooms. The rooms are colour coded so can be easily distinguishable.

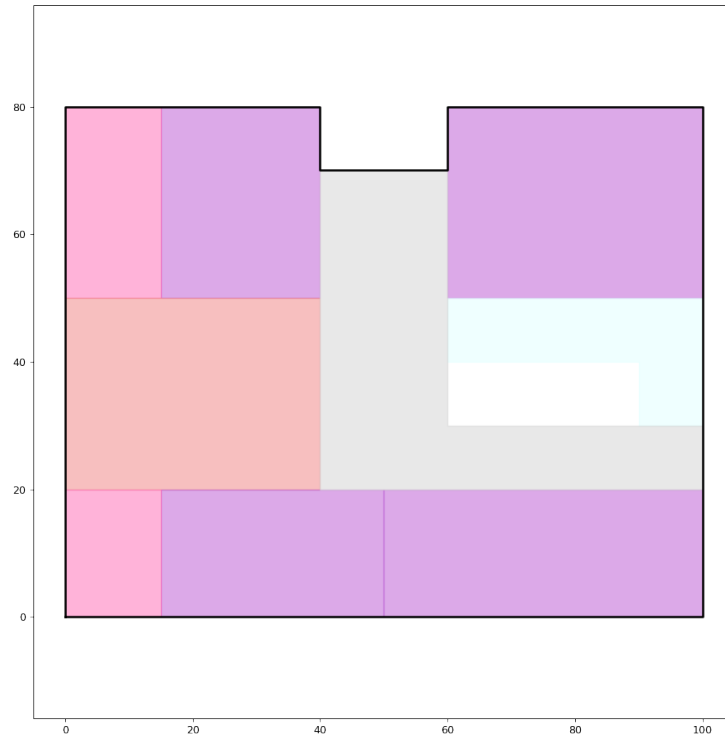


Figure 6.21: Rendered Floor Plan using Matplotlib

The shoelace area method uses the shoelace area theorem to calculate the area of a room given its coordinates. The formula for the shoelace area is as follows.

$$A = \frac{1}{2}(x_0y_1 - x_1y_0 + \dots + x_{n-2}y_{n-1} - x_{n-1}y_{n-2} + x_{n-1}y_0 - x_0y_{n-1})$$

where $(x_0, y_0), \dots, (x_{n-1}, y_{n-1})$ are the vertices of a polygon.

This allows us to effectively work out the area of any room, regardless of its shape.

The add room method works by identifying if there is any empty space in the floor plan, i.e. spaces in the floor plan where no rooms are in. If there is an empty space, the method adds a room to this space. If there is no empty space in the floor plan, then the method finds the largest room in the floor plan and splits it into two rooms.

The remove room method works by receiving an index for a room that needs to be removed and returns a new list with all rooms but the room with this index.

The find adjoining walls method works by receiving a room and a target room as inputs. The method works by searching through all walls in the original room and the target room and uses the shapely library to detect any intersection between these walls. This method will return a list of the coordinates of any (sections of) walls where an intersection has been detected.

The move wall method receives a list of room, a room index, a wall index, a direction and a shift magnitude as inputs. The room index refers to which room requires a wall shift. The wall index refers to which wall within a room requires a shift. The direction (up, down, right or left) indicates in what direction the wall shift will occur. The shift refers to by how much the wall will shift by.

The method first checks if the wall is bounded to the floor plan boundary. If the full length of the wall intersects with the floor plan boundary, we prevent the wall shift from taking place. If only a section of the wall intersects with the floor plan boundary, then we update the wall coordinates that we need to shift to the section which doesn't intersect. This allows portions of walls to be shifted if shifting the full wall will result in walls being shifted outside the floor plan boundary.

We then iterate through all walls in every rooms to see if the wall coordinates intersect with any other rooms. If the wall intersects with another, in part or in full, we store these coordinates. Here we also check for an intersection of the wall with the stairs and prevent moving this wall.

The method then performs a shift on the wall as per the direction and shift magnitude. The wall is only shifted in an orthogonal way (horizontal or vertical). If an attempt is made to shift a horizontal wall horizontally, the method prevents this so as to not create any diagonal walls. This is similarly done for vertical walls. If the wall shift causes a wall to go outside of the floor plan boundary, then the shift is rejected.

If a shift to a wall has been made, we examine the coordinates of intersecting walls stored earlier. If there are any intersecting walls with the original wall, then we move them in a similar way and create a connecting wall if needed to prevent gaps in the room.

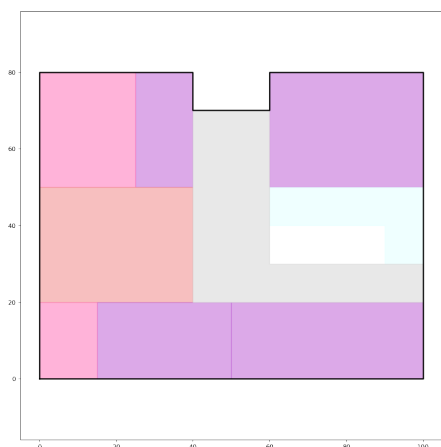


Figure 6.22: Moving a Wall (1)

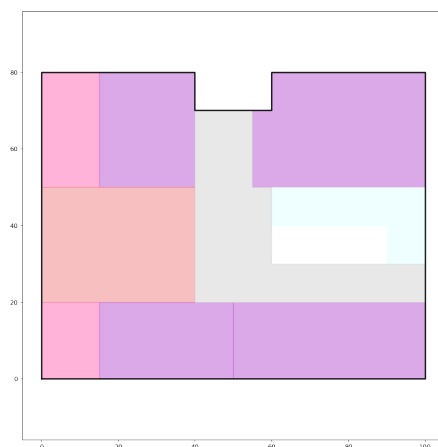


Figure 6.23: Moving a Wall (2)

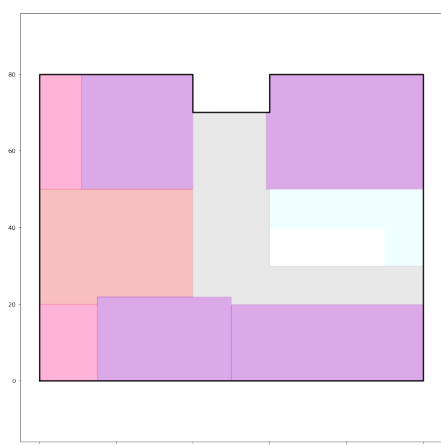


Figure 6.24: Moving a Wall (3)

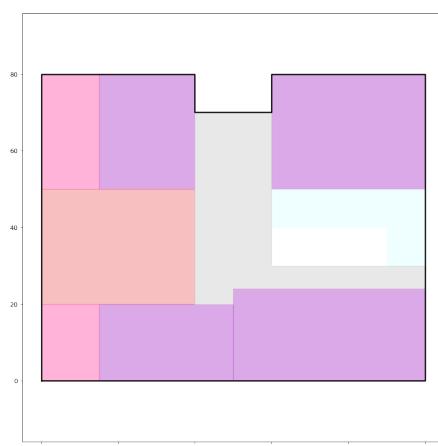


Figure 6.25: Moving a Wall (4)

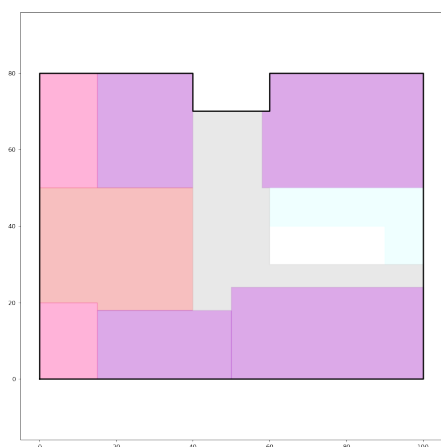


Figure 6.26: Moving a Wall (5)

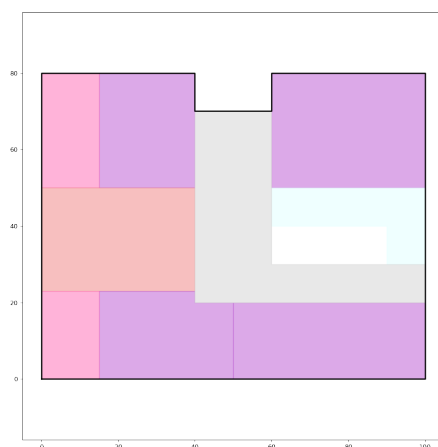


Figure 6.27: Moving a Wall (6)

Floor Plan Environment Class

The `FloorPlanEnv` class provides the main interface that trains the PPO agent. It inherits from the `Gymnasium` library and contains a pair of before and after floor plans, the current floor plan, the action space, the observation space, the maximum number of consecutive steps an agent can take, the current step it is on and the directions that a wall can be shifted in. It contains a step method, a method to calculate the reward of a step taken by the agent, a reset method to reinitialise the environment between episodes, a method to encode the floor plan in a way for the agent to understand and a method to calculate the similarity between two floor plans.

The step method uses the action sample created by the agent (facilitated by `Gymnasium`) to specify a room index, a wall index, a direction, and a shift. If the shift is greater than zero, this method runs the move wall function on the original floor plan and returns the rooms with any new coordinates. The current floor plan is then updated with these new room coordinates. If a change has been made, the method calls the calculate reward method. If a change has not been made, 0 is returned as the reward to disincentivise not making any changes. The current step is then incremented. Training is terminated after thirty steps of wall movements. This is to ensure a sufficient movement and exploration within a floor plan.

The calculate reward function works by first calculating the similarity between the newly generated floor plan after a wall shift and the original modified floor plan. It then calculates the proportional area of each room in the floor plan. This is calculated by dividing the total area of the floor plan, provided by the user, by the shoelace area of the floor plan. This allows us to get the relational area for one unit of the floor plan. Using this, the method checks to see if there are any bedrooms or kitchens below the specified minimum size. If there are, for every violation, the reward is deducted. This is to create a significant deterrent in making bedrooms and kitchens too small, and therefore not compliant to legal regulations. Furthermore, the method sums the total area for all rooms (excluding corridors and stairs) in the newly generated floor plan and the previous floor plan. If an increase in room space has been detected, a reward is added. This aims to favour maximising room sizes over corridor sizes. A ratio is also calculated by dividing the area of rooms by the total area of the floor plan space. Additionally, a method is created to calculate what walls have been displaced, if any, and assigns a cost proportional to the displacement. This simulates the cost minimisation by providing an increasingly negative

reward for a higher number of wall displacements, which represent extra material. Each of these rewards carry an associated weight. The final reward is calculated by adding the room-floor plan ratio (weighted 0.4), the similarity (weighted 0.2), the number of legal violations (weighted 0.2), the cost penalty (weighted 0.2) and the room growth reward. This aims to, with highest priority, optimise space, and balance out all other constraints afterwards.

The calculate similarity method works by checking the wall coordinates, room areas and placement similarities between two floor plans, with a higher weight placed on the wall similarity and then the room area similarity to generate a similarity score. To check wall coordinates, all walls are compared from one floor plan to the walls in the target floor plan and a minimum distance to a wall is calculated. This is considered it's best match. This exponential of this distance is used as a measure of similarity and is added to the total similarity. As this is done for all walls, the total similarity of the floor plans is calculated. The room areas are calculated using the shoelace area method and are compared. Finally this method uses the Jaccard similarity for walls. The Jaccard similarity refers to a measure of similarity between two sets. This is calculated by:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

where A is set 1 and B is set 2. In this method, set A refers to the grid of the original floor plan and set B refers to the grid of the target floor plan. These metrics are weight so that wall coordinates similarity is the most important, room area similarity is next important and grid similarity is the least important. Even if the trained floor plan isn't completely consistent with the target floor plan, it should follow a similar pattern in terms of these room areas and wall coordinates. That's why the grid similarity is weighted below the other two metrics.

The reset method sets the floor plan pairs to a randomly selected pair in the training set, and sets the current step back to 0 to essentially restart the training process.

Other methods are included to encode the environment in a format understandable by the agent. This is done by creating a grid corresponding to the coordinates of a floor plan space and assigning a number to each point corresponding to a room or the floor plan boundary. Whilst floor plan designs are plotted against a 100x100 grid to better facilitate small shifts as well as large shifts. The environment provided by Gymnasium limits an environment of 20x20.

To resolve this, the environment is first laid out as a 100x100 grid and once done, downsampled to the 20x20 size.

Single Floor Plan Environment Class

The `SingleFloorPlanEnv` class inherits from the `FloorPlanEnv` class and is the main class used in optimising floor plans from the web application. Since the web application does not provide a pair of floor plans but instead a singular floor plan that needs optimisation, this class is required. This class mimics the behaviour of the `FloorPlanEnv` class, but is specifically compatible with only one floor plan. It makes use of the methods in the `FloorPlanEnv`.

Training the Agent

To train the agent, vectorised environments are instantiated of the `FloorPlanEnv` class to facilitate running multiple simulations in parallel, allowing for accelerated learning. The vectorised environment also standardises observations and rewards by normalising and clipping the values. This prevents extreme observation and reward values. Vectorised environments are provided as part of the `stable_baselines3` library. An evaluation environment is also instantiated to monitor the performance of the agent. A model of the agent is then set up using the PPO library from `stable_baselines3`, defining its learning rate, number of steps, the batch size, number of training epochs, a discount factor, a factor for Generalised Advantage Estimation (GAE), a clipping parameter, a coefficient for entropy regularisation, a coefficient for the value function loss and a gradient clipping threshold. The training set and validation set is defined and the agent is trained using the `learn` command from `stable_baselines3` for a defined number of time steps. The final model is saved after training. After training, the agent is evaluated using an evaluation function. Here, a trained agent is run for multiple episodes to calculate average performance.

Learning Parameters

- Learning Rate - controls how much the policy is updated per optimisation step. The value chosen here is $3e-4$. With a lower learning rate, drastic updates are prevented, leading to more stable training. However, since it is really small, it can cause very slow convergence.
- Number of Steps - the number of steps collected before updating the policy. The value chosen was 1024. Whilst to optimise training, a higher value would be ideal in improving gradient estimates, this value presents a reasonable trade-off between efficiency and computational power needed.

- Batch Size - the number of samples processed in each optimisation step. The value chosen here was 128. Larger values would be ideal in providing more stable gradients by reducing the variance. The chosen value presents a reasonable trade-off between efficiency and computational power needed. Values that are too small lead to noisy updates.
- Number of Epochs - the number of passes through the collected data during each update. The value chosen was 10. Multiple passes extract more information from collected experiences, however, too many epochs can lead to overfitting. The chosen value favours sample efficiency and risks overfitting.
- Discount Factor - how much future rewards are discounted. The value chosen here was 0.99. This incentivises the agent to consider long-term rewards. This high value could lead to slow learning but is efficient at providing long-term results for sequential problems. A value too low could cause myopic behaviour.
- Generalised Advantage Estimation (GAE) - balances bias-variance trade-off in advantage estimation. A higher value here aims to decrease bias by increasing variance and a lower value vice-versa. The value chosen was 0.95 to smooth rewards while preserving long-term dependencies.
- Clip Range for Policy - limits policy updates to prevent destructive changes. This ensures stable training. The value chosen here was 0.2 to balance changes to stability whilst allowing for reasonable exploration.
- Normalise Advantage. This is set to true. It improves gradient stability at the cost of more computational overhead.
- Entropy Coefficient - encourages exploration by penalising low-entropy policies. This is set to 0.01. This value maintains moderate exploration without excessive randomness.
- Value Function Loss Weight - how much to weigh value loss versus policy loss. This aims to balance learning between the policy and value function. This is set to 0.5 to balance between neglecting policy updates and poor value estimates.
- Gradient Clipping - clips gradients to prevent exploding updates. This is set at 0.5 to stabilise training.
- Time Steps - total steps for the entire training run. Whilst a higher larger number of time steps produce better gradient estimates and improved exploration, it produces

significantly slower updates and memory overhead. To balance this, the time steps was set at 500,000.

- **Network Architecture** - defines the neural network architecture for policy and value functions. This has been set at three layers of 256 units to provide good depth for complex spatial relationships. This comes at the expense of longer training times.

Training Set

The training set for this agent is a number of pairs of before and after floor plan designs which have been synthetically generated to allow the learning of optimising space. The following figures represent some example of the synthetically produced floor plan pairs used for training.

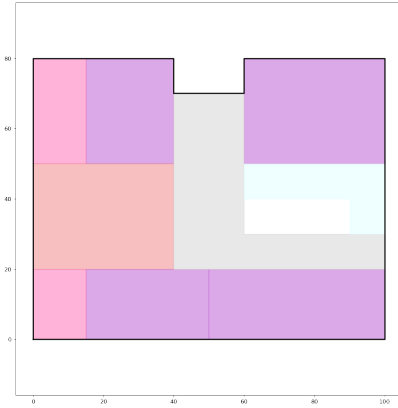


Figure 6.28: Training Set (1.1)

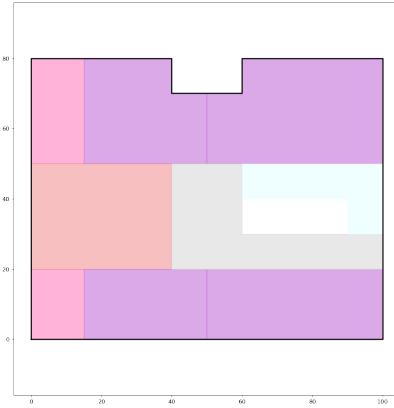


Figure 6.29: Training Set (1.2)

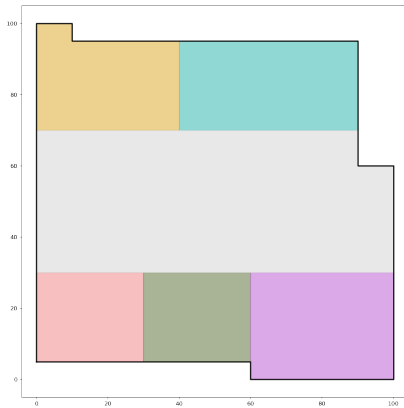


Figure 6.30: Training Set (2.1)

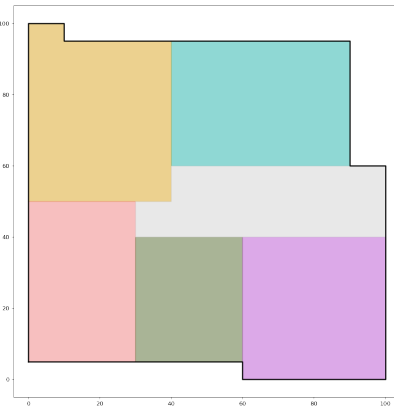


Figure 6.31: Training Set (2.2)

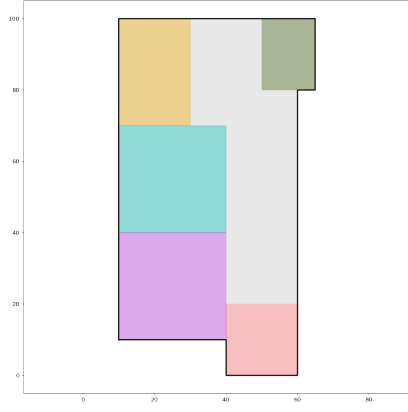


Figure 6.32: Training Set (3.1)

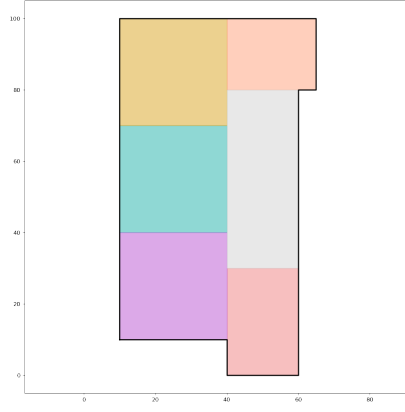


Figure 6.33: Training Set (3.2)

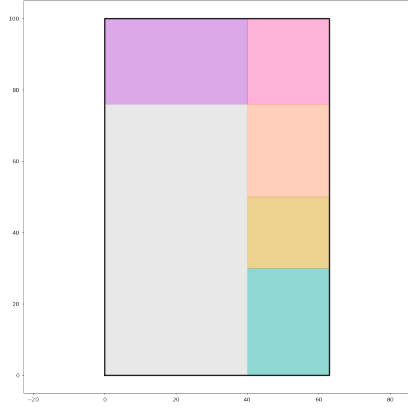


Figure 6.34: Training Set (4.1)

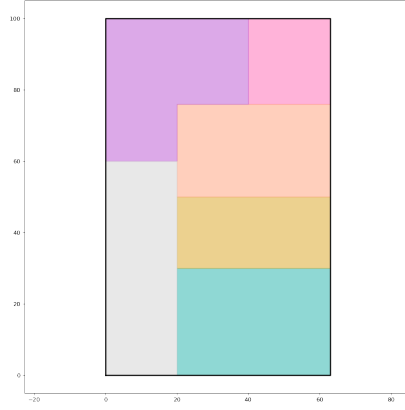


Figure 6.35: Training Set (4.2)

Testing the Agent

To test the learning of the agent, various metrics were logged during training.

- Mean Reward Per Episode - the average reward the agent receives per episode (one complete floor plan modification attempt). Whilst this can be quite noisy in early training due to increased exploration, this indicates how much the agent is able to maximise its rewards, and this improve space utility, reduce costs and adhere to legal constraints.
- Mean Squared Error (MSE) - the loss of the value function which predicts expected rewards. Low values of the MSE indicate more accurate value estimates and demonstrates how accurate the agent becomes at estimating over the number of time steps.

The trained model was additionally used on new sets of floor plan data. Examples of the results follow.

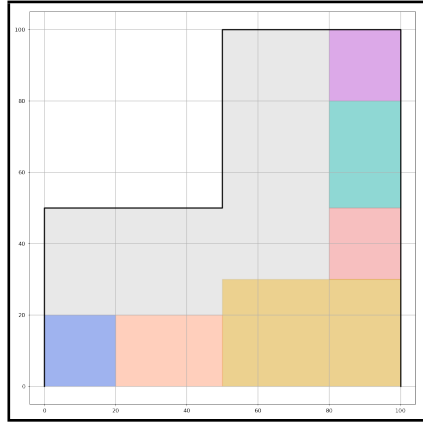


Figure 6.36: Testing Set (1)

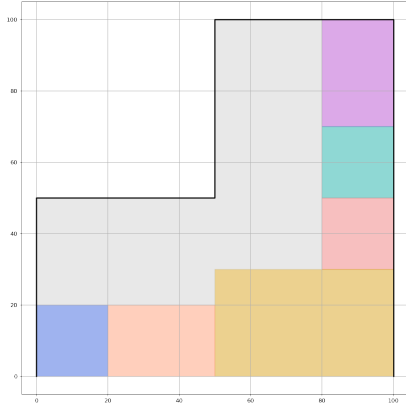


Figure 6.37: Testing Set (1.1)

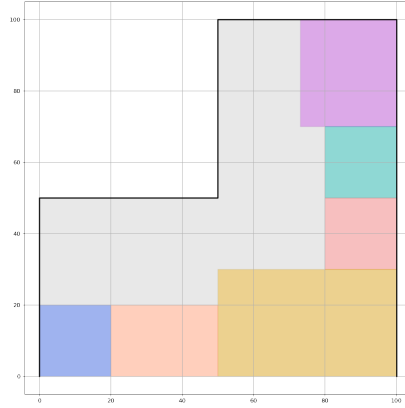


Figure 6.38: Testing Set (1.2)

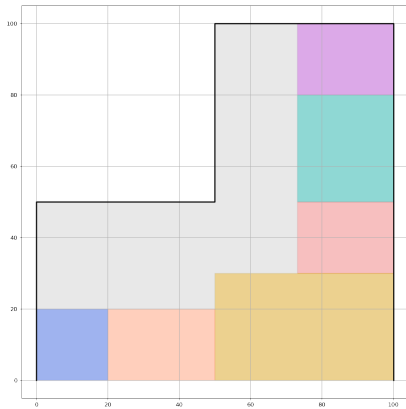


Figure 6.39: Testing Set (1.3)

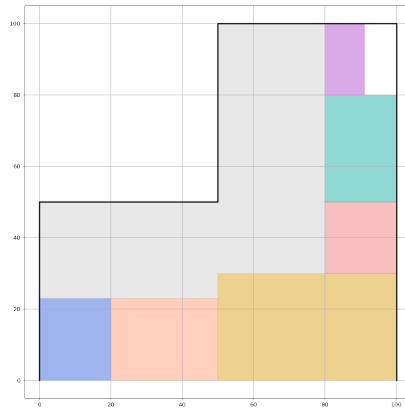


Figure 6.40: Testing Set (1.4)

In Figure 6.37: Testing Set (1.1), we see the agent enlarging the space of the bedroom (marked in purple) to ensure legal compliance. This is shown further in Figure 6.38: Testing Set (1.2).

In Figure 6.39: Testing Set (1.3), the agent enlarges the space of three rooms, successfully prioritising room space over corridor space, without completely eradicating the corridor. In Figure 6.40: Testing Set (1.4), there is an enlargement of the two rooms in the bottom left.

Whilst this shows initial success in optimising space, minimising costs and adhering to legal constraints, it's important to note that this is not always the case. In the following figures, the agent demonstrates a miscalculation in achieving these parameters, and therefore reduces overall space.

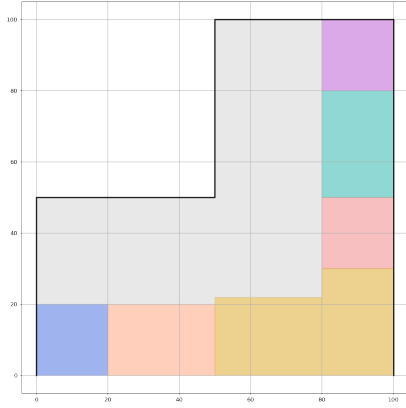


Figure 6.41: Testing Set (1.5)

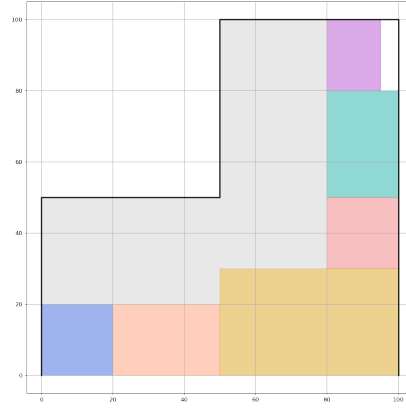


Figure 6.42: Testing Set (1.6)

During training, a checkpoint callback is established every 20,000 time steps, essentially allowing the progress of the agent to be tracked in this interval until all the time steps have concluded. At every interval, the performance is reviewed, and the model is compared with previous saved models to save the best performing model. This is established based on the mean reward per episode during that interval. Whilst also allowing logs to be kept for tracking and to prevent training from scratch in the event of a crash, we are able to extract the best performing agent over this 500,000 time step period, which may not necessarily be at the end.

Frontend:

The Floor Plans page allows a user to create a new floor plan and save it to a room. This is done by first drawing the boundary of the floor plan, submitting it and then drawing the rooms within the floor plan boundary. When this is done, the user can optimise the floor plan using the PPO agent and receive an optimised floor plan which they can save. As an extension to the design, for the user's convenience, the box which contains the floor plan drawing features a grid for precision and buttons have been added to hide or show the grid, to clear the boundary,

to clear the current room, to undo the last point, undo the last room and to reset all.

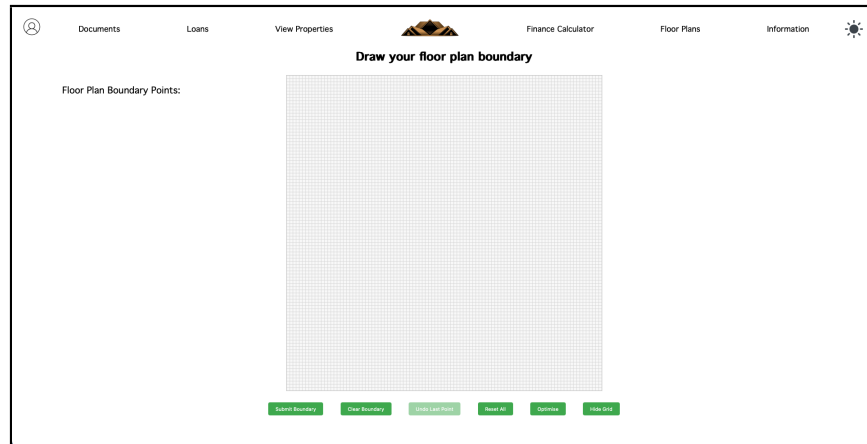


Figure 6.43: Floor Plans Page Implementation

To draw, the user essentially plots points on the grid. This has been modified to snap to only integer coordinates. To further ease the drawing process, the user is shown clear guidelines on where they can plot the next point and also keeps a record of the plotted coordinates for the user to see.

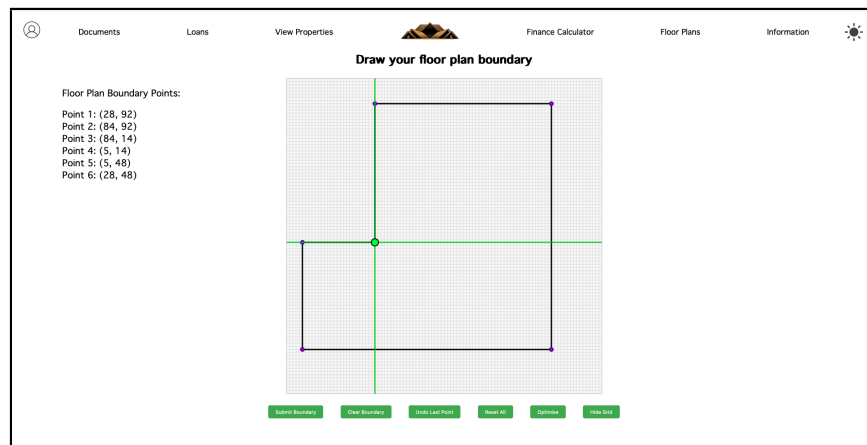


Figure 6.44: Drawing the Floor Plan Boundary

Once a floor plan boundary has been drawn, the user can select a room type and begin plotting points in a similar fashion. The user is shown the completed rooms and the points of the current room being drawn. They are shown similar guidelines for where they can plot points. When a room's points can be connected in a way which prevents diagonal walls, the border of the drawn shape will turn green, indicating a room can be submitted here. The rooms are labelled for the user's convenience. When a user has finished drawing a room, they can select the finish room

button which completes the room within the floor plan and allows them to draw another. This functionality has been safeguarded to prevent selecting points outside the floor plan boundary, within other rooms which would cause an overlap between two rooms and creating diagonal walls. The floor plan boundary and rooms must have a minimum of four points. If less points have been plotted when submitting, an error message is shown to the user.

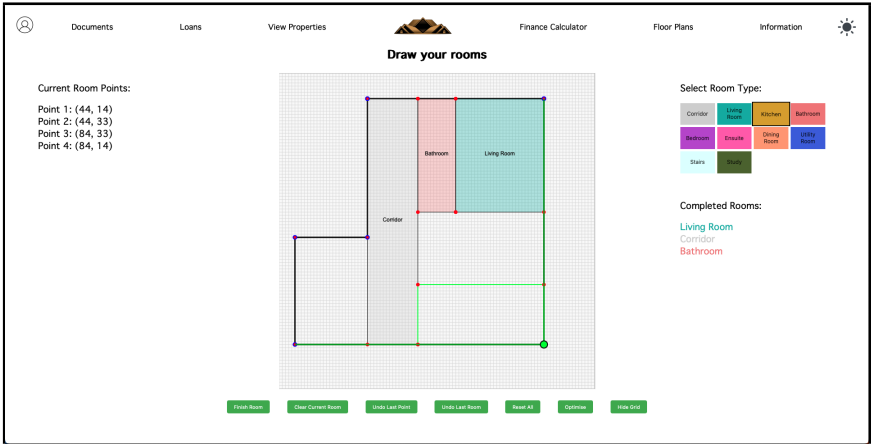


Figure 6.45: Drawing the Floor Plan Rooms (1)

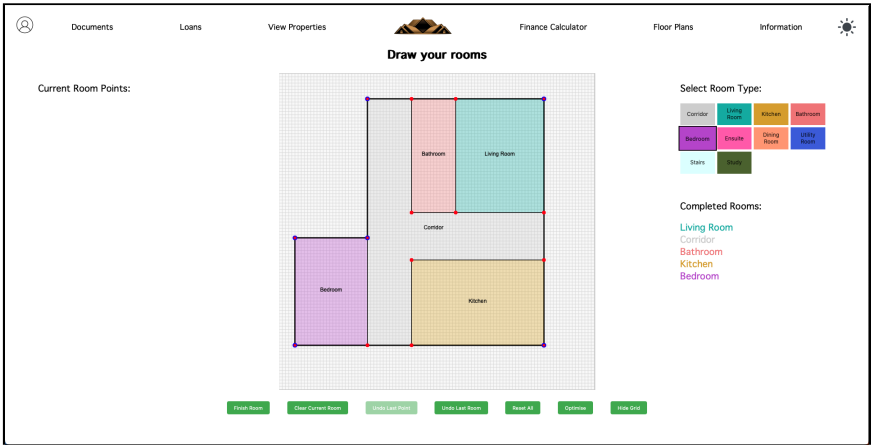


Figure 6.46: Drawing the Floor Plan Rooms (2)

When a user has finished drawing their floor plan and attempts to optimise the floor plan, they are prompted to enter the total area of the floor plan in square metres. This is important to ensure legal compliance (minimum room sizes) are met when optimising.

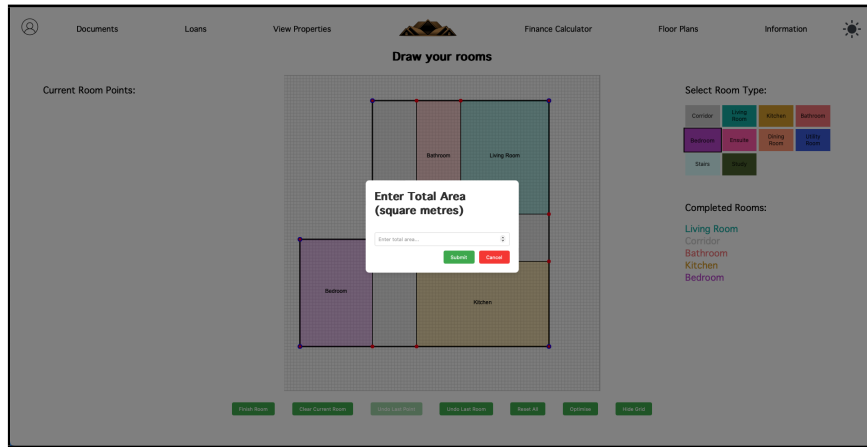


Figure 6.47: Entering the Floor Plan Area

Connecting the Frontend to the Backend:

Here, we attempt the communication between drawing an existing floor plan on the frontend and sending it to the backend to optimise and respond. We do this by creating a new method in the views.py file which imports the SingleFloorPlanEnv and creating a new environment with the boundary coordinates and room coordinates of the drawn floor plan. The received data is validated in this method and if valid, instantiates an instance of the FloorPlan class which is sent to the SingleFloorPlanEnv environment. Using the best saved model from training, the agent is then run for 1000 steps to optimise the floor plan. An example of the results is shown below.

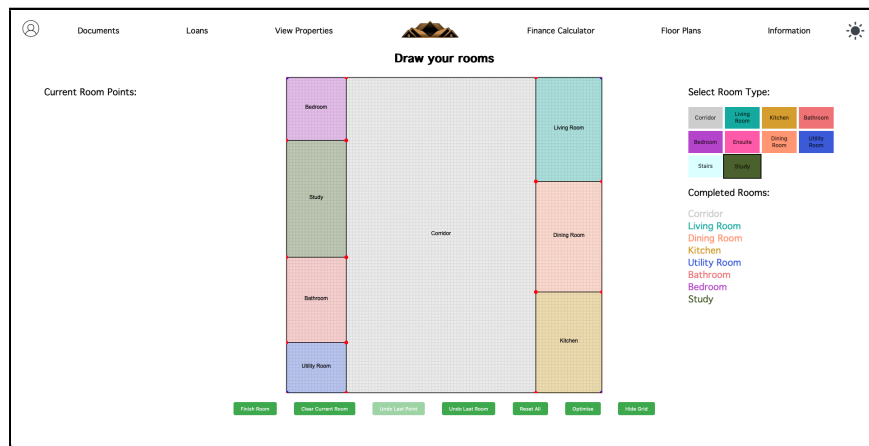


Figure 6.48: Optimising a Floor Plan - Drawing the Original Space

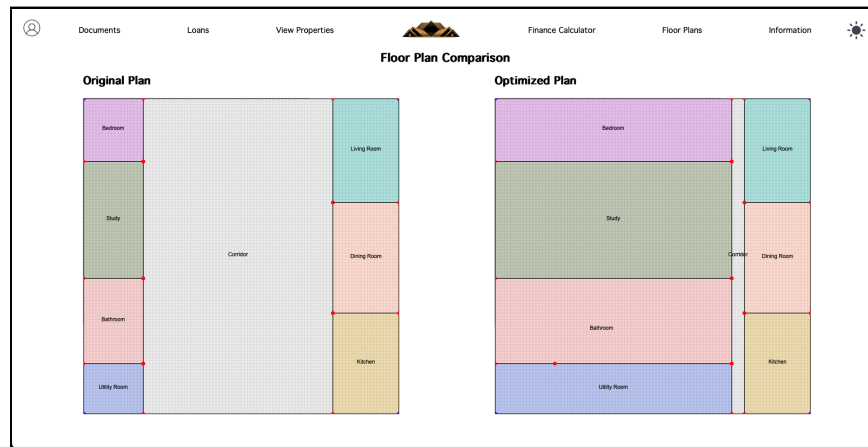


Figure 6.49: Optimising a Floor Plan - Displaying an Optimised Floor Plan

The result shown here shows an admirable at maximising space and increasing bedroom sizes to meet legal requirements. In terms of cost, its opted to shift four walls and extend three on the left rather than shifting three and extending two on the right. However, since the bedroom was not initially compliant and therefore necessitated a shift, it proceeded to naturally shift the remaining walls, and perhaps in this way minimised the cost but not at the expense of more space. These results could indicate a potential need to define a clear minimum width or area for the corridor to provide a more natural layout, but display a promising foundation for balancing the constraints of spatial utility, cost efficiency and legal compliance.

Chapter 7

Legal, Social, Ethical and Professional Issues

7.1 Legal and Ethical Issues

7.1.1 British Computer Society (BCS) Legislation

The British Computer Society (BCS) aims to set and maintain standards of competence, conduct and ethical practice within the Information Technology (IT) profession [48]. The BCS Code of Conduct [49], requires the following:

- Public interest - have due regard for public health, privacy, security, wellbeing, right, environment wellbeing, no discrimination and equal access.
- Professional competence and integrity - developing professional knowledge, skills and competence, ensuring legislation compliance, working ethically and respecting alternative viewpoints.
- Duty to relevant authority - carry out professional responsibilities with diligence, avoid conflict with the relevant authority and accepting professional responsibility.
- Duty to the profession - uphold the reputation of the profession, act with integrity and seek to improve professional standards.

The BCS Code of Good Practice offers good practices on project management, relationship management, security, safety, change and quality.

This project has been created with the aim of abiding by the BCS Code of Conduct and BCS Code of Good Practice.

7.1.2 UK Data Protection Legislation

Data protection legislation [50] aims to control how personal information is used by organisations. In the UK, data protection is governed by the UK GDPR and the Data Protection Act 2018. These laws require personal data to be used fairly, lawfully and transparently, used for specified, explicit purposes, used in a way that is adequate, relevant and limited to only what is necessary, to be accurate, up to date and kept for no longer than is necessary and to be handled in a way that ensures appropriate security, including protection against unlawful or unauthorised processing, access, loss, destruction or damage.

The secure storage and handling of data along with the rigorous authentication procedure used within this project aims to respect these data protection regulations.

7.2 Social and Professional Issues

Whilst this project presents a fundamental foundation in the optimisation of architectural design, there are many critical features that have not been explored. This includes factoring in accessibility for those with disabilities and promoting physical and psychological comfort through adequate natural light, ventilation and noise reduction. Some of these factors are additional legal requirements that need to be explored for professional use of this application. Professional use of this project would further require a factor of sustainability, aiming to minimise waste and promoting renewable materials and methods. Automating this optimisation process can cause a massive generalisation of floor plan design, potentially discarding important cultural aspects of a community or location.

7.3 Real Estate Legal Compliance

Whilst efforts have been made to consider a range of legal compliance regarding real estate, this project features an over-generalisation of the specific rules laid out by the relevant regulations and must not be used as an exact reference, but a guide to managing properties and optimising

floor plan space. Real estate regulations are constantly changing and could therefore mean that the parameters used in this project are no longer applicable in the future.

Chapter 8

Evaluation

8.1 PPO Agent

Based on the reward function configuration of the agent, the aim was to feed the trained agent a new floor plan, and it would shift walls in a way that optimises space, minimises cost and adheres to legal compliance. The results from testing the agent are as follows.

Mean Reward Per Episode

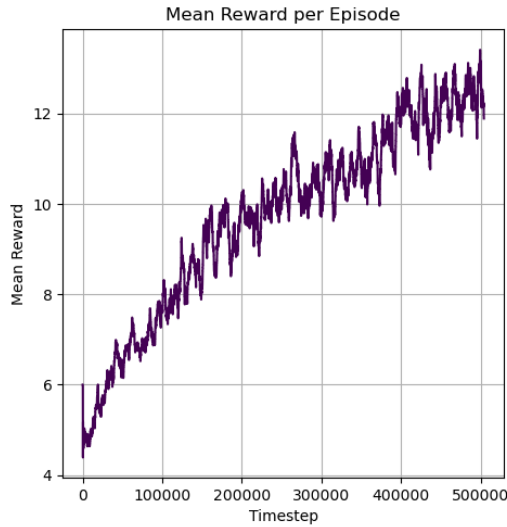


Figure 8.1: PPO Agent - Mean Reward Per Episode

This figure shows a proportional increase in the mean reward per episode over the duration of training. This indicates strong learning from the agent to select actions which maximise the

rewards and therefore fit the requirements of optimal space, minimal costs and legal adherence. There is a significant amount of noise during early training which demonstrates the agent's exploration early on, and exploitation later on. This is consistent with expectations.

Although the PPO agent demonstrates a high capacity in maximising rewards, due to its clipping mechanism to stabilise training and prevent large changes could be hindering its progress in learning how to maximise rewards quicker. Algorithms such as Q-Learning and MCTS could potentially result in faster reward maximisation due to their larger updates. With greater computation resources, perhaps these algorithms could've proved a better fit.

Mean Squared Error (MSE)

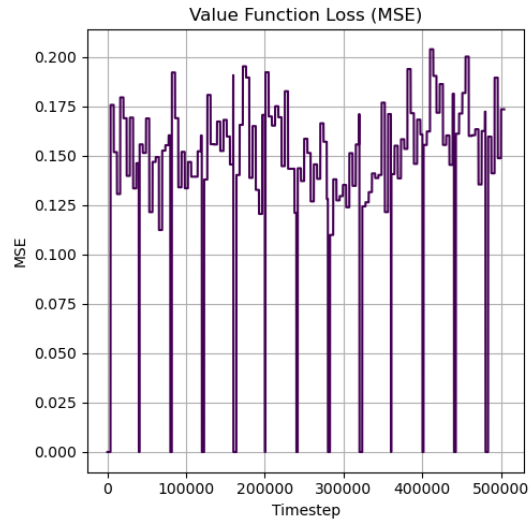


Figure 8.2: PPO Agent - MSE

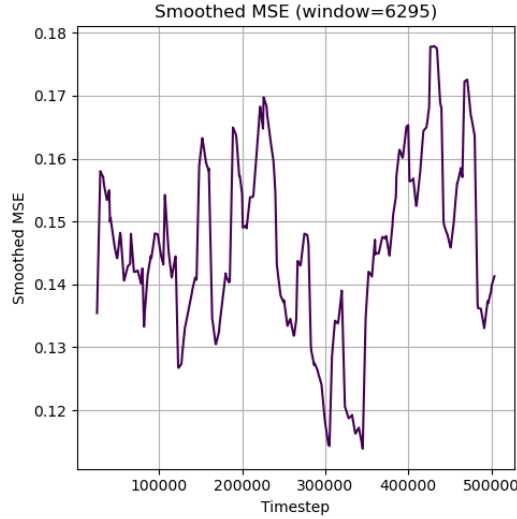


Figure 8.3: PPO Agent - Smoothed MSE

The MSE, discussed in Chapter 2, presents a trend of errors, i.e the difference between the observed values and the expected values. This result is much less consistent with expectations. The figures shown here shows a disproportional trend with the time steps. The MSE consistently increases and decreases dramatically over the time steps. This could perhaps be explained by a wide range of exploration done by the agent, which then allowed it to learn better ways of optimising, thus being able to bring the overall MSE down to a low value just past midway through training. However, past this point, the agent proceeds to achieve even higher rates of error than what was seen before this. The MSE is represented for PPO as the value loss - the error in the value function's predictions, indicating that the agent wasn't able to effectively predict the reward of being in a particular state after an action was chosen. Accurate value predictions are crucial for PPO's policy optimisation process, as they help the agent understand the value of different actions and states, ultimately leading to better policies. However, this could be justified by the relatively low number of floor plan training pairs available to the agent and a relatively low number of time steps needed for optimal training. Whilst further evidence would be needed to prove this, it can be assumed that more accurate training data and a higher number of time steps would help significantly in bringing down the MSE. In spite of this, the MSE revolves around a very narrow range and demonstrates huge potential for improvement.

Algorithms such as MADDPG, which makes use of multiple agents, may have presented better estimation of values and therefore a lower MSE. Additionally, since the observation space was hugely downsampled to fit into Gymnasium's environment, another custom environment or an environment with a higher observation space such as PettingZoo could've worked in con-

junction with MADDPG to support multi-agent learning. The use of a Convolutional Neural Networks (CNN) may have proved a worthy replacement for the currently implemented MLP policy, accommodating for high-dimensional data such as floor plans, especially with increasing constraints.

Overall, the agent shows a substantial capacity for learning to optimise floor plans based on the constraints of space utility, cost efficiency and legal and regulatory compliance, with a strong ability to maximise rewards. Being able to capture these constraints and translate them into higher rewards with the resources provided to it is nothing short of impressive.

Chapter 9

Conclusion

9.1 Closing Remarks

The use of Reinforcement Learning (RL) in architectural optimisation demonstrates a profound opportunity for architectural and real-estate development. Whilst prior methods have focused on generating new floor plans under various constraint, this project focused its investigation on modifying existing floor plans under the constraints of space, cost and legal compliance, specifically for real estate management. Using a Proximal Policy Optimisation (PPO) algorithm incited an exceptional balance between the three constraints - space utility, cost efficiency and legal and regulatory compliance.

Notable advances are made in the project in deepening the understanding of the relationship between these constraints and how they can work together to provide optimal floor plan designs. Additionally, this project improves our conceptual understanding of how we can use this optimisation of floor plan design to specifically induce a higher internal value of properties and maximise rental turnover, providing critical insights to real estate owner, managers and investors. It provides them with key insights on how to best manage their interior space. Whilst the scope of this project focuses on optimising architecture from a real estate management perspective, the use cases for this type of optimisation is abundant. The principles established in this report provides a fundamental foundation in architecture and space layout design and could benefit architects, construction workers, residential home owners, businesses, governments and designers.

Overall, the work undertaken by this project bridges theoretical architectural optimisation concepts with practical business outcomes, demonstrating an advancement in the use of com-

putational approaches for market advantages.

9.2 Future Work

Real estate management and architectural optimisation encompass a comprehensive range of sub-fields and processes which would be impossible to embrace within this project under the time constraint. Whilst the focus of this project was facilitating easy access to properties in one place and optimising floor plans based on space, cost and compliance, there are many further aspects which remain to be explored.

Agent Training

Whilst the agent trained in this report shows significant capability at learning to maximise reward by balancing the constraints, it is purely informed on synthetic data. There is not sufficient, high-quality data of before and after floor plans in the way this project required. For this reason, floor plan pairs had to be synthetically generated to simulate the action of optimising space, minimising costs and adhering to legal constraints. The training set was relatively limited to what is essential for optimal training of the agent. Ideally, there would be sufficient, real data that would be used for agent training. Regardless, the quality of the data is a more determining factor of the success of the agent's training and would require expert floor plan design modifications which strictly align with the given constraints to produce the best result. Despite this, the agent was still able to display notable results.

In addition to training data, it is essential to have a diverse range of valid testing data to better monitor the performance of the agent, identify potential weak spots and check for generalisation. This would provide crucial aspects on many fundamental parts of floor plan design that the agent has yet to take into account.

Another limiting factor in the training of the agent is in computational power and resources. The training of the agent to optimise floor plans with many distinct parameters constitutes a necessity for a large number of complex processes, time steps and simulations.

Real Estate Management

Whilst this project allows a user to keep a convenient record of their properties and relevant information, there are many more aspects to consider within real estate management. A more in depth analysis of the cash flow would provide substantial benefit to a real estate owner, manager or investor, tracking the real income that has been received and the real expenses

that have been incurred. Income could be calculated as gross income and net income as per the relevant tax regulations and expenses can be monitored as business or personal expenses. Additionally, it could prove beneficial for a user to have the ability to track any loans and mortgages they currently have, setting clear deductions from received income and clearly being displayed for the user. This could be further extended to forecast interest rates and other factors that may affect the loans or to calculate the total cost and interest of taking out a new loan. Currently, the user is required to manually enter all the information about properties, income and expenses. Perhaps it could be beneficial to explore a method to automate this process so as to provide more time and convenience for the user. This could be done by analysing bank records or other documents. Furthermore, as discussed previously, RL could be used in other aspects of real estate management such as to calculate property value and rent price trajectory. The functionality of the application could also be extended to further stakeholders such as tenants and investors, providing an online platform for all relevant tenancy contracts and transactions, or investments and returns.

Architectural Optimisation

This project focuses on optimising floor plans based on space, costs and legal compliance. However, there are many more parameters that encompass the true functionality of a floor plan. As explored by ArchiGAN, incorporating a style parameter within the floor plan can allow users to maintain a certain culture in their space. Similarly, accounting for the connectivity of rooms and the circulation within a floor plan can better place rooms within a floor plan which follow natural human patterns and optimise for convenience. Additionally, it could prove beneficial to explore the use of angled (or even semi-round walls) within a floor plan to optimise space, as explored in SpaceLayoutGym. By doing this, floor plans could perhaps better place en-suites and other complementary rooms without excessively taking up the space of larger room. There should also be some account of the structural integrity of a building, for example, load-bearing walls to provide optimisations that can actually work in practice. Whilst legal compliance has been explored in terms of minimum room sizes, the values used in this project are a major simplification of the regulations outlined by different housing authorities. Ideally, this project would be able to recognise the location of a property and access the relevant housing authority regulations and extract the exact standards needed for the property. Since, this is done on a location basis, it requires extensive research into the standards of all areas. This could extend to global use, which would require even further research of compliance. In extension to mini-

mum room sizes, there are general rules for fire safety, fire escapes, amenities and cleanliness. Perhaps these are factors that can also be explored by an agent. This project could further be extended to allow functionality for fenestration (window placement) and entrance placement, as well as calculate associated costs for moving, adding or removing windows and entrances. Finally, the design of the agent could be refined to allow for modifications of the exterior floor plan boundary, which could be useful for attaining planning permission and creating extensions to properties.

Whilst this report explored an in-depth perspective of architectural optimisation within a real estate management environment, adjusting for spatial utility, cost efficiency and legal compliance, it redefines the use of Reinforcement Learning (RL) in enhancing and automating traditionally human-driven processes.

References

- [1] Paul Tostevin, Charlotte Rushton - Savills Research, “Total value of global real estate: Property remains the world’s biggest store of wealth.” [Online] <https://www.savills.com/impacts/market-trends/the-total-value-of-global-real-estate-property-remains-the-worlds-biggest-store-ofwealth.html>. (Accessed on 26/03/2025).
- [2] Statista, “Real estate - worldwide.” [Online] <https://www.statista.com/outlook/fmo/real-estate/worldwide?currency=GBP>. (Accessed on 26/03/2025).
- [3] Liam Bailey, Sunny Creative, “The wealth report - knight frank,” *The Wealth Report*, vol. 19, p. 30, 2025.
- [4] MFS UK, “How many houses are in the uk and where could you look for opportunity?.” [Online] <https://www.mfsuk.com/blog/how-many-houses-in-the-uk>. (Accessed on 27/03/2025).
- [5] Costmodelling, “Uk tender price and building cost indices.” [Online] <https://costmodelling.com/construction-indices>. (Accessed on 27/03/2025).
- [6] GOV.UK, “Selective licensing in the private rented sector: a guide for local authorities.” [Online] <https://www.gov.uk/government/publications/selective-licensing-in-the-private-rented-sector-a-guide-for-local-authorities/selective-licensing-in-the-private-rented-sector-a-guide-for-local-authorities>. (Accessed on 28/03/2025).
- [7] GOV.UK, “Houses in multiple occupation and residential property licensing reform: guidance for local housing authorities.” [Online] <https://www.gov.uk/government/publications/houses-in-multiple-occupation-and-residential-property-licensing-reform-guidance-for-local-housing-authorities/houses-in-multiple-occupatio>

n-and-residential-property-licensing-reform-guidance-for-local-housing-authorities. (Accessed on 28/03/2025).

- [8] IBM, “What is a neural network?.” [Online] <https://www.ibm.com/think/topics/neural-networks>. (Accessed on 02/04/2025).
- [9] Kusum Saini, “Types of cyber attacks explained.” [Online] <https://www.simplilearn.com/tutorials/cyber-security-tutorial/types-of-cyber-attacks>. (Accessed on 31/03/2025).
- [10] Hazelcast, “What is a web session?.” [Online] <https://hazelcast.com/foundations/caching/web-session>. (Accessed on 31/03/2025).
- [11] Subham Datta, Michal Aibin, “What are sessions? how do they work?.” [Online] <https://www.baeldung.com/cs/web-sessions>. (Accessed on 31/03/2025).
- [12] mdn web docs, “Working with json.” [Online] https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Scripting/JSON. (Accessed on 31/03/2025).
- [13] GeeksforGeeks, “Json web token (jwt).” [Online] <https://www.geeksforgeeks.org/json-web-token-jwt>. (Accessed on 31/03/2025).
- [14] Okta, “Json web token (jwt): A special form of auth token.” [Online] <https://www.okta.com/uk/identity-101/what-is-token-based-authentication>. (Accessed on 31/03/2025).
- [15] GeeksforGeeks, “Access token vs refresh token: A breakdown.” [Online] <https://www.geeksforgeeks.org/access-token-vs-refresh-token-a-breakdown>. (Accessed on 31/03/2025).
- [16] Cloudflare, “What is transport layer security (tls)?.” [Online] <https://www.cloudflare.com/en-gb/learning/ssl/transport-layer-security-tls>. (Accessed on 31/03/2025).
- [17] Cloudflare, “What is ssl?.” [Online] <https://www.cloudflare.com/en-gb/learning/ssl/what-is-ssl>. (Accessed on 31/03/2025).
- [18] Cloudflare, “Http vs. https: What are the differences?.” [Online] <https://www.cloudflare.com/en-gb/learning/ssl/why-is-http-not-secure>. (Accessed on 31/03/2025).
- [19] Mariko Kosaka, “Same-origin policy.” [Online] <https://web.dev/articles/same-origin-policy>. (Accessed on 31/03/2025).

- [20] Mariko Kosaka, “Cross-origin resource sharing (cors).” [Online] <https://web.dev/articles/cross-origin-resource-sharing>. (Accessed on 31/03/2025).
- [21] Stanislas Chaillou, “Ai + architecture - towards a new approach,” Master’s thesis, Harvard GSD, 2019.
- [22] Reza Kakooe, Benjamin Dillenburger, “Illuminating spaces deep reinforcement learning and laser-wall partitioning for architectural layout generation,” Master’s thesis, ETH Zurich, 2025.
- [23] React Documentation, “Getting started with react.” [Online] <https://reactjs.org/docs/getting-started.html>. (Accessed on 29/03/2025).
- [24] Vite.js Documentation, “Why vite.” [Online] <https://vitejs.dev/guide/why.html>. (Accessed on 29/03/2025).
- [25] Django, “Django documentation.” [Online] <https://docs.djangoproject.com/>. (Accessed on 30/03/2025).
- [26] Farama Foundation, “Gymnasium documentation.” [Online] <https://gymnasium.farama.org>. (Accessed on 30/03/2025).
- [27] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, “Stable-baselines3: Reliable reinforcement learning implementations,” *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.
- [28] John Hunter, Darren Dale, Eric Firing, Michael Droettboom, “Matplotlib.” [Online] <https://matplotlib.org>. (Accessed on 30/03/2025).
- [29] Farama Foundation, “Sean gillies and shapely contributors.” [Online] <https://shapely.readthedocs.io/en/stable>. (Accessed on 30/03/2025).
- [30] GeeksforGeeks, “How to create a basic api using django rest framework?” [Online] <https://www.geeksforgeeks.org/how-to-create-a-basic-api-using-django-rest-framework>. (Accessed on 31/03/2025).
- [31] Awofesobi kolade Peace, “Axios in react js.” [Online] <https://medium.com/@awofesobipeace/axios-in-react-js-f5a0cd821694>. (Accessed on 31/03/2025).
- [32] Lucidchart, “Types of uml diagrams.” [Online] <https://www.lucidchart.com/blog/types-of-UML-diagrams>. (Accessed on 30/03/2025).

- [33] Josip Vojak, “Exploring the pros and cons of sql databases.” [Online] <https://medium.com/@josipvojak/exploring-the-pros-and-cons-of-sql-databases-mysql-postgres-oracle-microsoft-sql-and-amazon-3c8de880b8d4>. (Accessed on 31/03/2025).
- [34] GeeksforGeeks, “Mongodb advantages & disadvantages.” [Online] <https://www.geeksforgeeks.org/mongodb-advantages-disadvantages>. (Accessed on 31/03/2025).
- [35] GeeksforGeeks, “Introduction to sqlite.” [Online] <https://www.geeksforgeeks.org/introduction-to-sqlite>. (Accessed on 31/03/2025).
- [36] Code Review Doctor, “Hacking django websites: Man in the middle attack.” [Online] <https://codereviewdoctor.medium.com/hacking-django-websites-man-in-the-middle-attack-7d3dce8e827d>. (Accessed on 31/03/2025).
- [37] Daniel Doody, “Best practices for securing django rest framework apis.” [Online] <https://medium.com/@daniel.doody/best-practices-for-securing-django-rest-framework-apis-9dbb7ba367a8>. (Accessed on 31/03/2025).
- [38] Jacob Kaplan-Moss, “Preventing sql injection in django.” [Online] <https://jacobian.org/2020/may/15/preventing-sqli>. (Accessed on 31/03/2025).
- [39] Code Review Doctor, “Hacking django websites.” [Online] <https://codereviewdoctor.medium.com/hacking-django-websites-6002241ef2ea>. (Accessed on 31/03/2025).
- [40] mdn web docs, “Cross-site request forgery (csrf) prevention.” [Online] https://developer.mozilla.org/en-US/docs/Web/Security/Practical_implementation_guides/CSRF_prevention. (Accessed on 31/03/2025).
- [41] Osirus Djodji, “Handling cors in django rest framework: A comprehensive guide.” [Online] <https://medium.com/django-unleashed/handling-cors-in-django-rest-framework-a-comprehensive-guide-ec11b0bc6807>. (Accessed on 31/03/2025).
- [42] Mehedi Khan, “Securing django rest apis with jwt authentication using simple-jwt.” [Online] <https://medium.com/django-unleashed/securing-django-rest-apis-with-jwt-authentication-using-simple-jwt-a-step-by-step-guide-28efa84666fe>. (Accessed on 31/03/2025).
- [43] Sarah Laoyan, “What is agile methodology? (a beginner’s guide).” [Online] <https://asana.com/resources/agile-methodology>. (Accessed on 01/04/2025).

- [44] npmjs, “dotenv).” [Online] <https://www.npmjs.com/package/dotenv>. (Accessed on 01/04/2025).
- [45] Sanket Shah, “Mastering environment variables: Best practices for npm dotenv usage).” [Online] <https://www.dhiwise.com/post/environment-variables-best-practices-for-npm-dotenv-usage>. (Accessed on 01/04/2025).
- [46] Ihor Lukianov, “Python — environment variables using dotenv library).” [Online] <https://lukianovihor.medium.com/python-environment-variables-using-dotenv-library-71529ad0e9c3>. (Accessed on 01/04/2025).
- [47] Django REST framework, “Settings).” [Online] <https://www.django-rest-framework.org/api-guide/settings>. (Accessed on 01/04/2025).
- [48] BCS, “Our royal charter).” [Online] <https://www.bcs.org/about-us/our-royal-charter>. (Accessed on 02/04/2025).
- [49] BCS, “Code of conduct for bcs members).” [Online] <https://www.bcs.org/about-us/our-royal-charter>. (Accessed on 02/04/2025).
- [50] GOV.UK, “Data protection).” [Online] <https://www.gov.uk/data-protection>. (Accessed on 02/04/2025).

Appendix A

Extra Information

List of Figures

3.1	ArchiGAN Three-Step Generation Stack	20
3.2	SpaceLayoutGym Dynamic Planning Process	21
5.1	System Architecture Diagram	31
5.2	Use Case Diagram	32
5.3	UML Diagram	33
5.4	Home Page Design	35
5.5	Login Page Design	35
5.6	Sign Up Page Design	36
5.7	Dashboard Page Design	37
5.8	User Dropdown Design	37
5.9	View Properties Page Design - Selecting a Portfolio	38
5.10	Creating a Portfolio	38
5.11	Editing a Portfolio	38
5.12	Deleting a Portfolio	38
5.13	View Properties Page Design - Selecting a Property	39
5.14	View Properties Page Design - Viewing a Property	39
5.15	Creating a Property	40
5.16	Editing a Property	40
5.17	Deleting a Property	40
5.18	Creating a Room	41
5.19	Editing a Room	41
5.20	Deleting a Room	41
5.21	Floor Plans Page Design	42
5.22	Floor Plans Page Design - Optimisation View	42
5.23	Floor Plans Page Design - Saving a Floor Plan	42

6.1	Frontend Folder Structure	48
6.2	Backend Folder Structure	51
6.3	Login Page Implementation	59
6.4	Sign Up Page Implementation	60
6.5	Home Page Implementation	61
6.6	View Properties Page Implementation	64
6.7	View Properties Page - Dark Mode	64
6.8	View Properties Page - Hovering Over a Portfolio	65
6.9	View Properties Page - Editing a Portfolio	65
6.10	View Properties Page - Deleting a Portfolio	65
6.11	View Properties Page - Creating a Portfolio	66
6.12	View Properties Page - View Properties in Portfolio	67
6.13	View Properties Page - Hover Over a Property	67
6.14	View Properties Page - Creating a Property	67
6.15	View Properties Page - Editing a Property	68
6.16	View Properties Page - Deleting a Property	68
6.17	View Properties Page - Viewing a Property	69
6.18	View Properties Page - Adding a Room	69
6.19	View Properties Page - Editing Rooms	69
6.20	Dashboard Page Implementation	70
6.21	Rendered Floor Plan using Matplotlib	72
6.22	Moving a Wall (1)	74
6.23	Moving a Wall (2)	74
6.24	Moving a Wall (3)	74
6.25	Moving a Wall (4)	74
6.26	Moving a Wall (5)	74
6.27	Moving a Wall (6)	74
6.28	Training Set (1.1)	79
6.29	Training Set (1.2)	79
6.30	Training Set (2.1)	79
6.31	Training Set (2.2)	79
6.32	Training Set (3.1)	80
6.33	Training Set (3.2)	80

6.34 Training Set (4.1)	80
6.35 Training Set (4.2)	80
6.36 Testing Set (1)	81
6.37 Testing Set (1.1)	81
6.38 Testing Set (1.2)	81
6.39 Testing Set (1.3)	81
6.40 Testing Set (1.4)	81
6.41 Testing Set (1.5)	82
6.42 Testing Set (1.6)	82
6.43 Floor Plans Page Implementation	83
6.44 Drawing the Floor Plan Boundary	83
6.45 Drawing the Floor Plan Rooms (1)	84
6.46 Drawing the Floor Plan Rooms (2)	84
6.47 Entering the Floor Plan Area	85
6.48 Optimising a Floor Plan - Drawing the Original Space	85
6.49 Optimising a Floor Plan - Displaying an Optimised Floor Plan	86
8.1 PPO Agent - Mean Reward Per Episode	90
8.2 PPO Agent - MSE	91
8.3 PPO Agent - Smoothed MSE	92

Appendix B

User Guide

B.1 Installation

To use this application, you must install the necessary packages. This is included within the requirements.txt file.

To do this, begin by navigating to the terminal or command prompt and initiate a virtual environment.

On Mac, this is done by running:

```
virtualenv venv  
source venv/bin/activate
```

On Windows, this is done by running:

```
virtualenv venv  
.\venv\Scripts\activate.bat
```

You can install the relevant dependencies in the requirements.txt by navigating to the root folder of the project and running:

```
pip install -r ./requirements.txt
```

Once dependencies have been installed, you must initialise the database. This is done within the root directory of the project by:

```
python3 manage.py makemigrations  
python3 manage.py migrate
```

You are now ready to deploy the server. To do this we initialise the frontend and the backend separately. First navigate to the "propertymanager" directory from the root directory using:

```
cd propertymanager
```

And then initialising the backend server:

```
python3 manage.py runserver
```

Once this is complete, open a new terminal window, navigate to the frontend folder within the root directory of the project using: `cd frontend`

And initialise the frontend server:

```
npm run dev
```

You should now be ready to use the application. Once both servers are running, the application will be available at `localhost:5173` in your web browser.